Introduction to ggplot2. Descriptive statistics I

Molecular Approaches to Biodiversity and Ecology

Nasta Lianguzova

Faculty of Biology, Shenzhen MSU-BIT University

2026-11-02

Today you will be able to

- Recognize tidy data and turn a wide table into a long one
- Read an Excel sheet into R and check it for missing values
- Build a plot with `ggplot2` layer by layer: data, aesthetics, geoms
- Show groups with color, shape and facets; use a logarithmic axis
- Draw histograms and save a plot to a file
- Describe a sample with the median, quartiles and the five-number summary; draw and read boxplots

Tidy data

Tidy data: find the problems

Up to 80% of the time of a data analysis goes into getting the data in order. R and `ggplot2` work best with **tidy data**.

	A	B	C	D	E	F	G
1	**Mitochondrial sequencing, Daphnia ponds, autumn 2026**						
2	**Sample**	**Collection**		**Sequencing**			**Notes**
3		**Date**	**Pond**	**Reads (M)**	**Length**	**GC**	
4	D1	02.11.2026	Pond A	12.5 / 13.1	16.5 kb	38%	
5		02.11.2026	Pond A	11.8	16540	0.37	re-run
6	D2	Nov 3	pond a		16 540 bp	37.9	
7		Nov 3	Pond A	no data	n/a	-	failed?
8	D3	2026-11-04	Pond B	9.6	16.6 kb	38 %	
9		2026-11-04	Pond B	0	-		contaminated
10	D4	4/11	B	10.2	16552	0.38	
11		4/11	B	10,9	16.55	38.2	
12	**TOTAL**			**54.4**			

A messy spreadsheet (messy_sheet_example.xlsx): how many problems can you find?

Spreadsheets: what not to do (structure)

Don't	Do instead
Merge cells (e.g. one sample ID across two rows)	One value per cell: repeat the ID in every row
Put a title or two header rows above the table	One header row; put titles and notes in a separate sheet
Put two values in one cell (`12.5 / 13.1`)	One value per cell: a second measurement is a new row
Code groups with cell colour	Put the group in its own column (e.g. `pond`); R cannot see colours
Add totals or means inside the data	Keep only raw data; calculate summaries in R

Each row is one observation, each column is one variable.

Spreadsheets: what not to do (values)

Don't	Do instead
Write units in the cells (`16.5 kb`, `38 %`)	Numbers only; the unit goes in the column name (`length_kb`, `gc_percent`)
Mix formats in one column (`38%`, `0.37`, `37.9`)	One format per column
Use several codes for missing (`no data`, `n/a`, `-`, `0`)	One code: `NA` or an empty cell. Never `0`: 0 is a value
Spell categories differently (`Pond A`, `pond a`, `B`)	One spelling for each category
Mix date formats (`02.11.2026`, `Nov 3`, `4/11`)	One format: `2026-11-04` (year-month-day)
Use a decimal comma (`10,9`)	A decimal point (`10.9`)
Type gene names into Excel without checking	Excel turns some gene names into dates (SEPT2 -> `2-Sep`): format the column as text

Wide and long format

Wide and long format

Remember the count matrix from last time? A **read count** is the number of sequencing reads that map to a gene: more reads means more expression.

```
counts <- matrix(c(1520, 980, 12, 1385, 1043, 30, 1610, 875, 7),
                 nrow = 3, ncol = 3)
rownames(counts) <- c("ACTB", "GAPDH", "MYC")
colnames(counts) <- c("sample_1", "sample_2", "sample_3")
counts_wide <- data.frame(gene = rownames(counts), counts)
counts_wide
##        gene sample_1 sample_2 sample_3
## ACTB   ACTB     1520     1385     1610
## GAPDH GAPDH      980     1043      875
## MYC     MYC       12       30        7
```

The gene names are now both row names (left) and a column, gene. Functions like `pivot_longer()` and `ggplot()` use only columns.

- **Wide format**: one row per gene, one column of counts per sample. Count tables usually arrive like this.
- **Long format**: one row per measurement (gene + sample + count). `ggplot2` and most statistical functions need it.

From wide to long: pivot_longer()

`pivot_longer()` from the `tidyr` package (part of the tidyverse) turns columns into rows. Its arguments:

- `cols`: which columns to turn into rows. `-gene` means "all columns except gene": the minus excludes a column.
- `names_to`: the name of the new column that gets the old column names (here, the sample names).
- `values_to`: the name of the new column that gets the values (here, the counts).

Each gene appears once for each sample: 3 genes × 3 samples = 9 rows. `pivot_wider()` does the reverse.

pivot_longer(): the long table

```
library(tidyr)
counts_long <- pivot_longer(counts_wide, cols = -gene,
                            names_to = "sample", values_to = "count")
counts_long
## # A tibble: 9 x 3
##   gene  sample   count
##   <chr> <chr>    <dbl>
## 1 ACTB  sample_1  1520
## 2 ACTB  sample_2  1385
## 3 ACTB  sample_3  1610
## 4 GAPDH sample_1   980
## 5 GAPDH sample_2  1043
## 6 GAPDH sample_3   875
## 7 MYC   sample_1    12
## 8 MYC   sample_2    30
## 9 MYC   sample_3     7
```

Your turn 1: wide to long

Read counts (made-up) of three genes in two control samples and two heat-stressed samples. *ACTB* and *GAPDH*, the housekeeping genes, stay stable. *HSPA1A* codes for HSP70, a heat-shock protein: cells make much more of it when they are stressed by heat.

```
heat_counts <- data.frame(gene = c("ACTB", "GAPDH", "HSPA1A"),
                          ctrl_1 = c(1490, 1012, 38),
                          ctrl_2 = c(1532, 968, 45),
                          heat_1 = c(1475, 1001, 1842),
                          heat_2 = c(1508, 990, 2115))
```

1. Turn `heat_counts` into a long table `heat_long` with the columns `gene`, `sample` and `count`.
2. How many rows does `heat_long` have, and why?

Your turn 1: answers

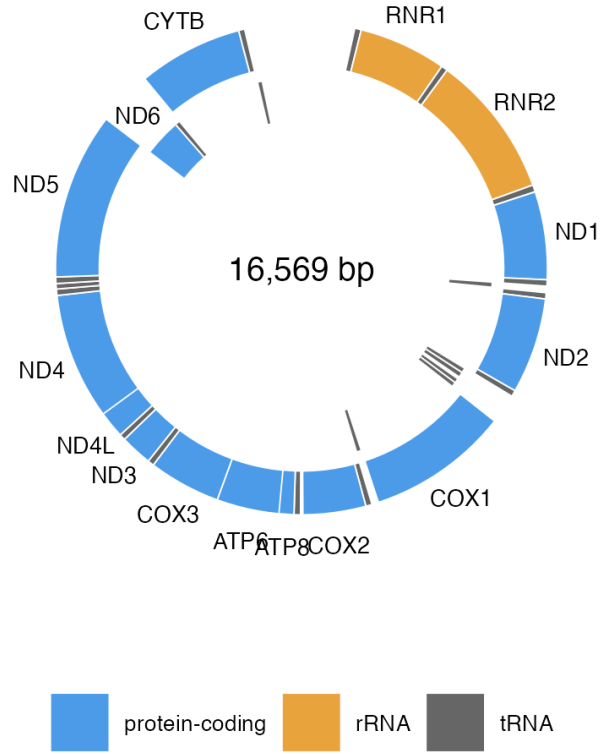
```
heat_long <- pivot_longer(heat_counts, cols = -gene,
                          names_to = "sample", values_to = "count")
heat_long
## # A tibble: 12 x 3
##    gene   sample count
##    <chr>  <chr>  <dbl>
##  1 ACTB   ctrl_1  1490
##  2 ACTB   ctrl_2  1532
##  3 ACTB   heat_1  1475
##  4 ACTB   heat_2  1508
##  5 GAPDH  ctrl_1  1012
##  6 GAPDH  ctrl_2   968
##  7 GAPDH  heat_1  1001
##  8 GAPDH  heat_2   990
##  9 HSPA1A ctrl_1    38
## 10 HSPA1A ctrl_2    45
## 11 HSPA1A heat_1  1842
## 12 HSPA1A heat_2  2115
nrow(heat_long)  # 3 genes x 4 samples = 12
## [1] 12
```

Our data: organelle genomes

Organelle genomes from NCBI RefSeq

`organelle_genomes.xlsx`: 150 complete genomes from NCBI RefSeq, 25 per group: mitochondria of 5 groups and plastids of land plants.

Column	What it is
`accession`	RefSeq number(s)
`organism`	species name
`group`	group of organisms
`organelle`	mitochondrion, plastid
`length_kb`	genome length, kb
`gc_percent`	GC content, %
`protein_genes`	protein-coding genes
`trna_genes`	tRNA genes
`note`	unusual records



NCBI gives lengths in bp; 1 kb = 1,000 bp, 1 Mb = 1,000 kb.

The human mitogenome (NC_012920.1): genes on the two DNA strands.

O'Leary N.A. et al. (2016). Reference sequence (RefSeq) database at NCBI. *Nucleic Acids Research* 44: D733–D745.

Reading an Excel sheet

Save `organelle_genomes.xlsx` into the `data` folder of your project. The package `readxl` (installed with the tidyverse) reads Excel files:

```
library(readxl)
genomes <- read_excel("data/organelle_genomes.xlsx",
                      sheet = "genomes", # sheet name
                      skip = 1,          # skip the "Source" row
                      na = "NA")         # code for missing values
```

- Without `skip = 1`, R would take the "Source: ..." row as column names.
- Without `na = "NA"`, the text `"NA"` would turn the number columns into text.

Checking the data

```
str(genomes)
## tibble [150 x 9] (S3: tbl_df/tbl/data.frame)
##  $ accession    : chr [1:150] "NC_083080.1" "NC_086888.1" "NC_027671.1"
↪  "NC_033386.1" ...
##  $ organism     : chr [1:150] "Bothus robinsi" "Choerodon azurio"
↪  "Cornufer vitianus" "Eleginops maclovinus" ...
##  $ group        : chr [1:150] "vertebrates" "vertebrates" "vertebrates"
↪  "vertebrates" ...
##  $ organelle    : chr [1:150] "mitochondrion" "mitochondrion"
↪  "mitochondrion" "mitochondrion" ...
##  $ length_kb    : num [1:150] 17.7 16.7 17.1 17 16.7 17 16.6 16.6 16.4
↪  16.5 ...
##  $ gc_percent   : num [1:150] 51.1 46.9 40.3 46.2 38.7 47.4 46.3 44.5
↪  32.9 46 ...
##  $ protein_genes: num [1:150] 13 13 13 13 13 13 13 13 13 13 ...
##  $ trna_genes   : num [1:150] 22 22 23 22 22 22 22 22 22 22 ...
##  $ note         : chr [1:150] NA NA "two tRNA-Met annotated; may be a
↪  real duplication" NA ...
```

`read_excel()` returns a tibble. `group` and `organelle` are text (`chr`); turn them into factors:

```
genomes$group <- factor(genomes$group)
genomes$organelle <- factor(genomes$organelle)
```

Unusual records: the note column

8 records have a `note`. Read it before you analyse the data:

- Split genomes: in *Hydra vulgaris*, *Spizellomyces punctatus*, *Cucumis sativus*, the mitogenome is split into several chromosomes. Each species is one row: the lengths are summed.
- Missing annotation: some records lack tRNA genes or annotate only part of the genome (next slides).
- Unusual counts: two records have more tRNA genes than their relatives; the note says why (later today).

```
genomes[!is.na(genomes$note), c("organism", "note")]
## # A tibble: 8 x 2
##   organism                note
##   <chr>                   <chr>
## 1 Cornufer vitianus       two tRNA-Met annotated; may be a real dupli~
## 2 Anemonia sulcata        tRNA genes not annotated in this record
## 3 Heteractis aurora       tRNA genes not annotated in this record
## 4 Hydra vulgaris          summed 2 chromosomes
## 5 Porites lutea           tRNA count unusual for a scleractinian cora~
## 6 Spizellomyces punctatus summed 2 chromosomes
## 7 Cucumis sativus         summed 3 chromosomes; only chromosome 1 ann~
## 8 Potentilla micrantha    tRNA genes not annotated in this record
```

Missing values: counting them

`is.na()` returns TRUE for each missing value; `sum()` of a logical vector counts the TRUE values:

```
sum(is.na(genomes$trna_genes))
## [1] 4
```

`summary()` describes each column; for numeric columns it also counts the NA's:

```
summary(genomes[, c("length_kb", "gc_percent", "protein_genes",
↪  "trna_genes")])
##    length_kb        gc_percent    protein_genes     trna_genes
##  Min.   :  14.8   Min.   :16.80   Min.   :13.00   Min.   : 1.00
##  1st Qu.:  16.6   1st Qu.:28.85   1st Qu.:13.00   1st Qu.:21.00
##  Median :  22.8   Median :37.50   Median :14.00   Median :22.00
##  Mean   : 131.8   Mean   :35.71   Mean   :27.44   Mean   :19.97
##  3rd Qu.: 154.9   3rd Qu.:43.90   3rd Qu.:35.00   3rd Qu.:24.00
##  Max.   :1684.6   Max.   :51.10   Max.   :81.00   Max.   :33.00
##                                   NA's   :1       NA's   :4
```

Missing values: which rows?

`complete.cases()` returns TRUE for each row without any NA:

```
nrow(genomes[!complete.cases(genomes), ])
## [1] 146
```

Too many: `note` is empty (NA) in 142 rows, and an empty note is not missing data. Check only the measurement columns:

```
num_cols <- c("length_kb", "gc_percent", "protein_genes", "trna_genes")
genomes[!complete.cases(genomes[, num_cols]), c("organism",
↪  "protein_genes", "trna_genes")]
## # A tibble: 4 x 3
##   organism             protein_genes trna_genes
##   <chr>                        <dbl>      <dbl>
## 1 Anemonia sulcata                13         NA
## 2 Heteractis aurora               13         NA
## 3 Cucumis sativus                 NA         NA
## 4 Potentilla micrantha            29         NA
```

Why each value is missing

The `note` column gives the reason for each record:

- *Anemonia sulcata* (cnidarians, mitochondrion): tRNA genes not annotated in this record.
- *Heteractis aurora* (cnidarians, mitochondrion): tRNA genes not annotated in this record.
- *Cucumis sativus* (land plants, mitochondrion): summed 3 chromosomes; only chromosome 1 annotated, gene counts NA.
- *Potentilla micrantha* (land plants, mitochondrion): tRNA genes not annotated in this record.

Two different reasons:

- The annotation is missing: the genes exist, but the record does not list them.
- Only part of the genome is annotated (the cucumber: 1 of its 3 chromosomes): a count would be too low.

In both cases the true number is unknown, so it is `NA`.

NA is not 0

The record of the sea anemone *Anemonia sulcata* (NC_049065.1) lists 13 protein-coding genes but no tRNA genes, while 22 of the 25 cnidarians in the table have 1 or 2. "Not annotated" does not mean "no genes". What if we wrote 0 instead of NA?

```
cnid <- genomes[genomes$group == "cnidarians", ]
mean(cnid$trna_genes, na.rm = TRUE)  # NA left out
## [1] 1.782609
trna_zero <- cnid$trna_genes
trna_zero[is.na(trna_zero)] <- 0     # wrong: NA written as 0
mean(trna_zero)
## [1] 1.64
```

Writing 0 for a missing value changes every statistic you calculate. `na.rm = TRUE` leaves the NA out instead.

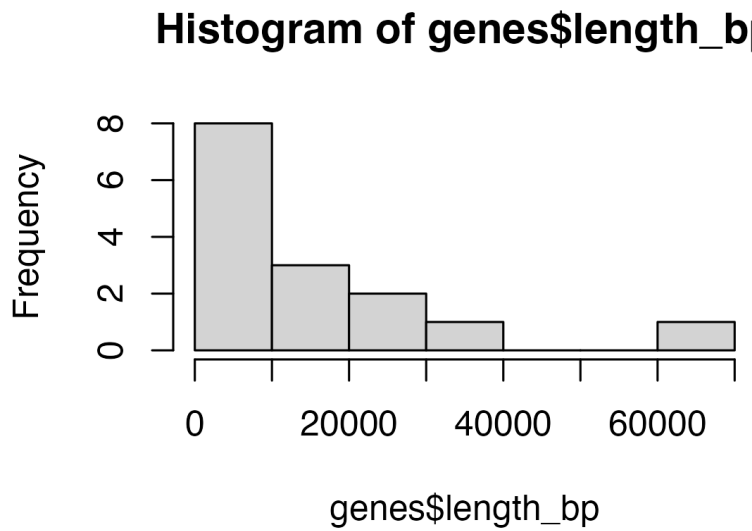
Plots with ggplot2

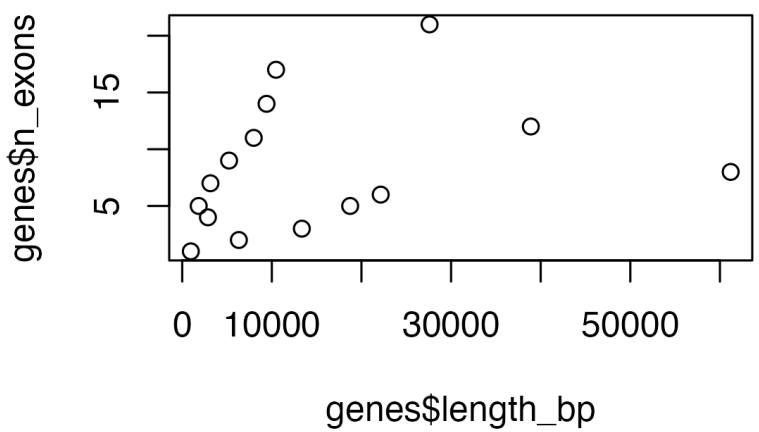
Last time: quick-look plots in base R

At the end of the last session we drew two quick plots of `genes.csv`:

```
genes <- read.csv("data/genes.csv")
hist(genes$length_bp)
```

```
plot(x = genes$length_bp,
     y = genes$n_exons)
```

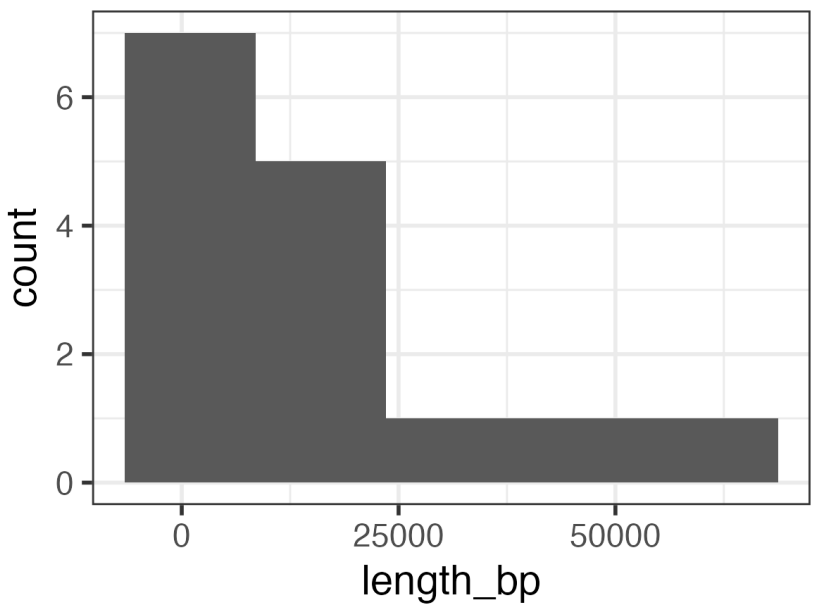




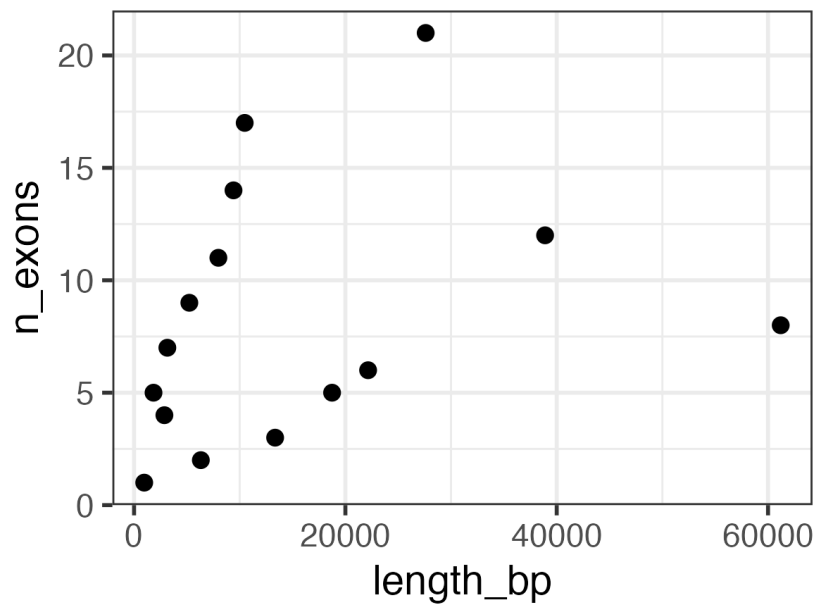
The same plots with ggplot2

`ggplot2` is a package from the tidyverse for publication-quality plots. Load it before the first plot:

```
library(ggplot2)
ggplot(data = genes,
       aes(x = length_bp)) +
  geom_histogram(bins = 5)
```




```
ggplot(data = genes,
       aes(x = length_bp,
           y = n_exons)) +
  geom_point()
```



How ggplot2 builds a plot

A ggplot is built from layers joined with +:

- `ggplot(data = genes)`: the data, a data frame (one column per variable).
- `aes(x = length_bp, y = n_exons)`: the **aesthetics**: which variable goes on which visual property (x, y, color, shape, size).
- `geom_point()`: the **geom**, the kind of object that draws the data (points, bars, boxes, lines).
- More layers change the look: axis labels, colors, themes, panels.

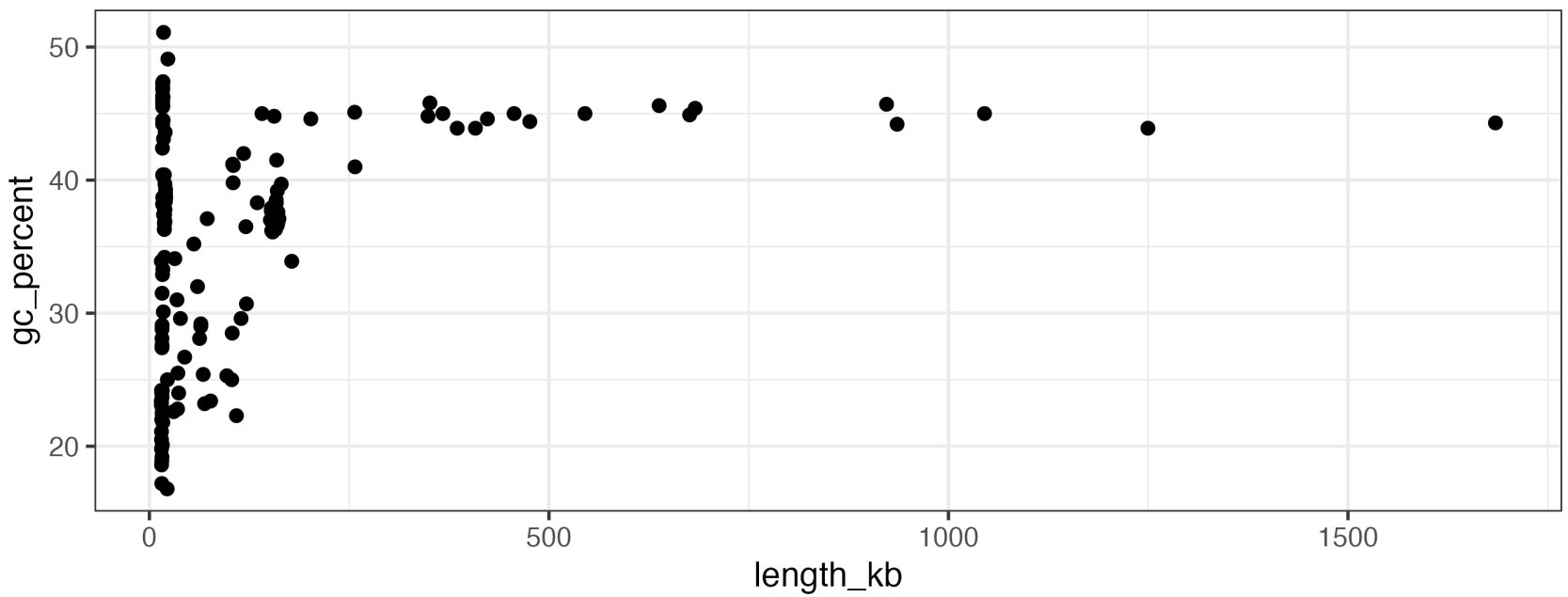
The idea comes from the "grammar of graphics" (Wilkinson 2005): every plot is data + aesthetics + geoms. `ggplot2` (gg = grammar of graphics) puts it into R.

Wickham H. (2016). *ggplot2: Elegant Graphics for Data Analysis*. Springer. Free online: https://ggplot2-book.org.

A first scatterplot

A **scatterplot** shows two numeric variables: genome length on x, GC content on y.

```
ggplot(data = genomes, aes(x = length_kb, y = gc_percent)) +
  geom_point()
```



Almost all points are squeezed against the left side.

Logarithms

The **logarithm** log10 answers "10 to what power gives this number?": $10^1 = 10$, $10^2 = 100$, $10^3 = 1000$, so log10(100) = 2 and log10(1000) = 3.

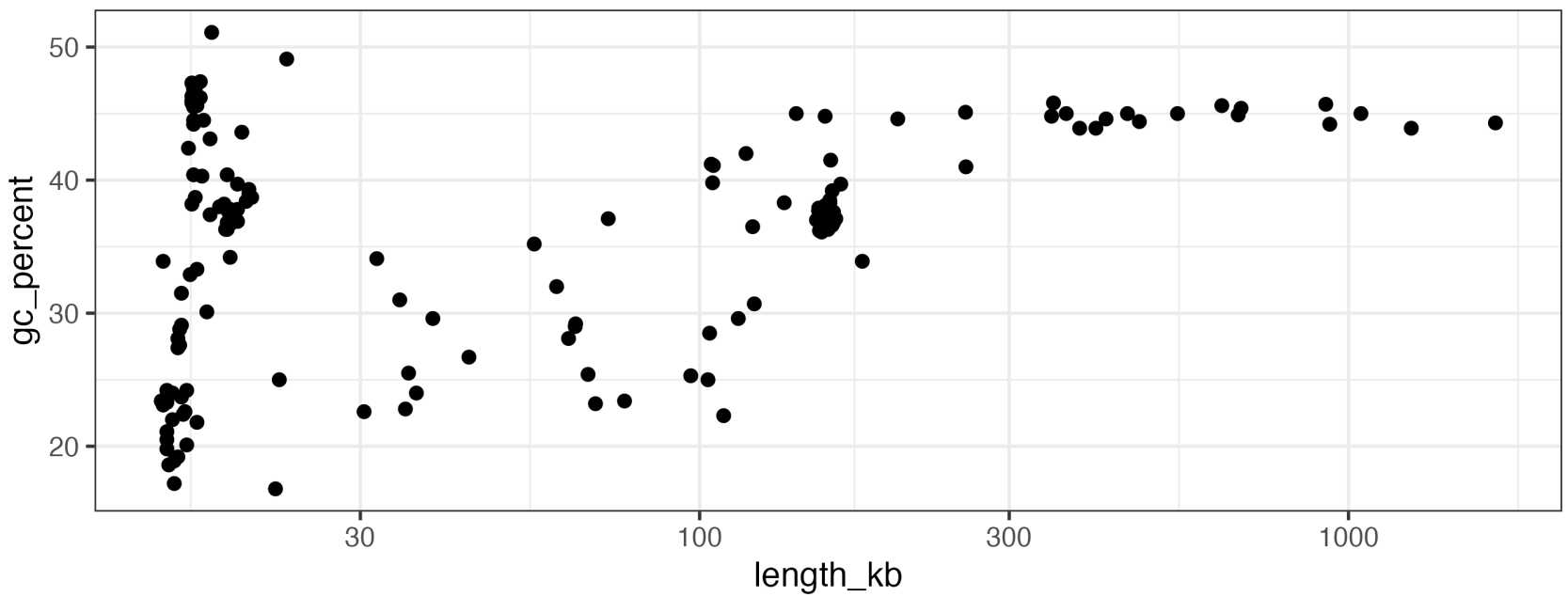
```
10^2
## [1] 100
log10(c(10, 100, 1000))
## [1] 1 2 3
log10(c(16, 160, 1600))  # each 10 times longer: +1
## [1] 1.20412 2.20412 3.20412
```

A step of 1 in log10 means "10 times more". Our genome lengths go from 14.8 to 1684.6 kb, so their log10 goes only from 1.17 to 3.23.

Why a logarithmic scale?

The largest genome is about 114 times the smallest. On a normal axis, all animal mitogenomes (14.8 to 23.1 kb) are squeezed into one corner. On a **logarithmic scale**, each step multiplies the value by 10 (10, 100, 1000 kb): small and large genomes get the same amount of space.

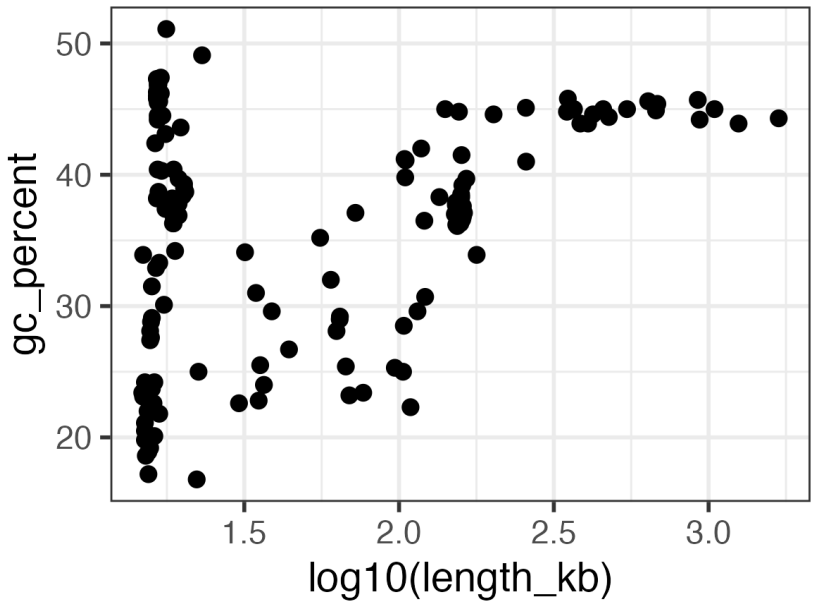
```
ggplot(data = genomes, aes(x = length_kb, y = gc_percent)) +
  geom_point() +
  scale_x_log10()
```



Transform the data or the axis?

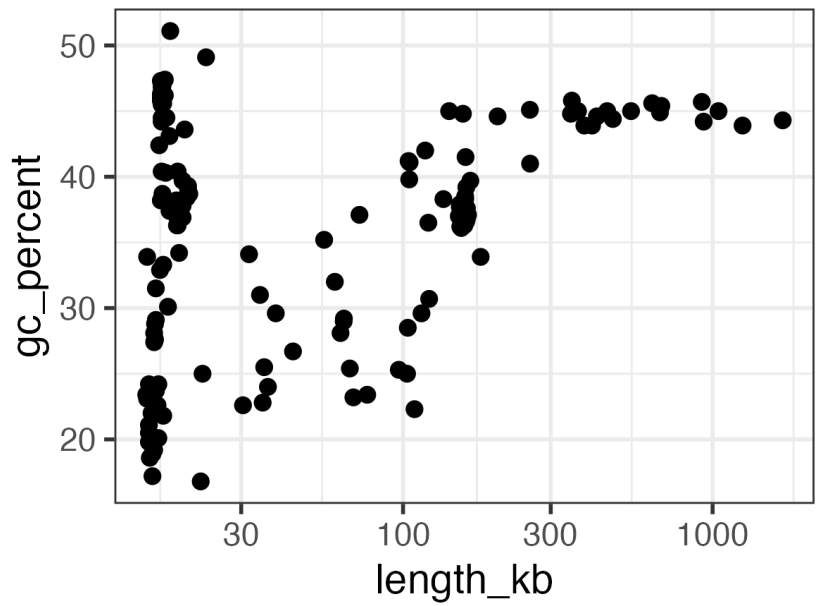
Transform the data: the axis shows log10 values.

```
ggplot(genomes,
       aes(x = log10(length_kb),
           y = gc_percent)) +
  geom_point()
```



Transform the axis: the labels stay in kb.

```
ggplot(genomes,
       aes(x = length_kb,
           y = gc_percent)) +
  geom_point() +
  scale_x_log10()
```

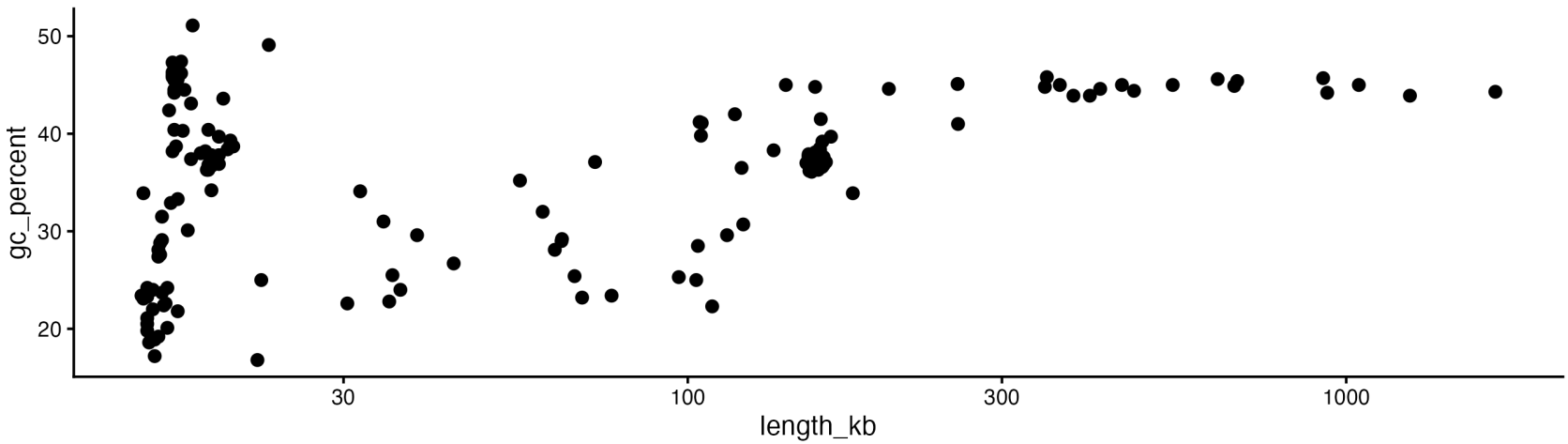


The points are in the same places. For plots, `scale_x_log10()` is easier to read: 2.0 on the left plot means 100 kb.

Themes

A theme changes the look of the plot, but not the data. Try `theme_bw()`, `theme_classic()`, `theme_minimal()`:

```
ggplot(data = genomes, aes(x = length_kb, y = gc_percent)) +
  geom_point() +
  scale_x_log10() +
  theme_classic()
```

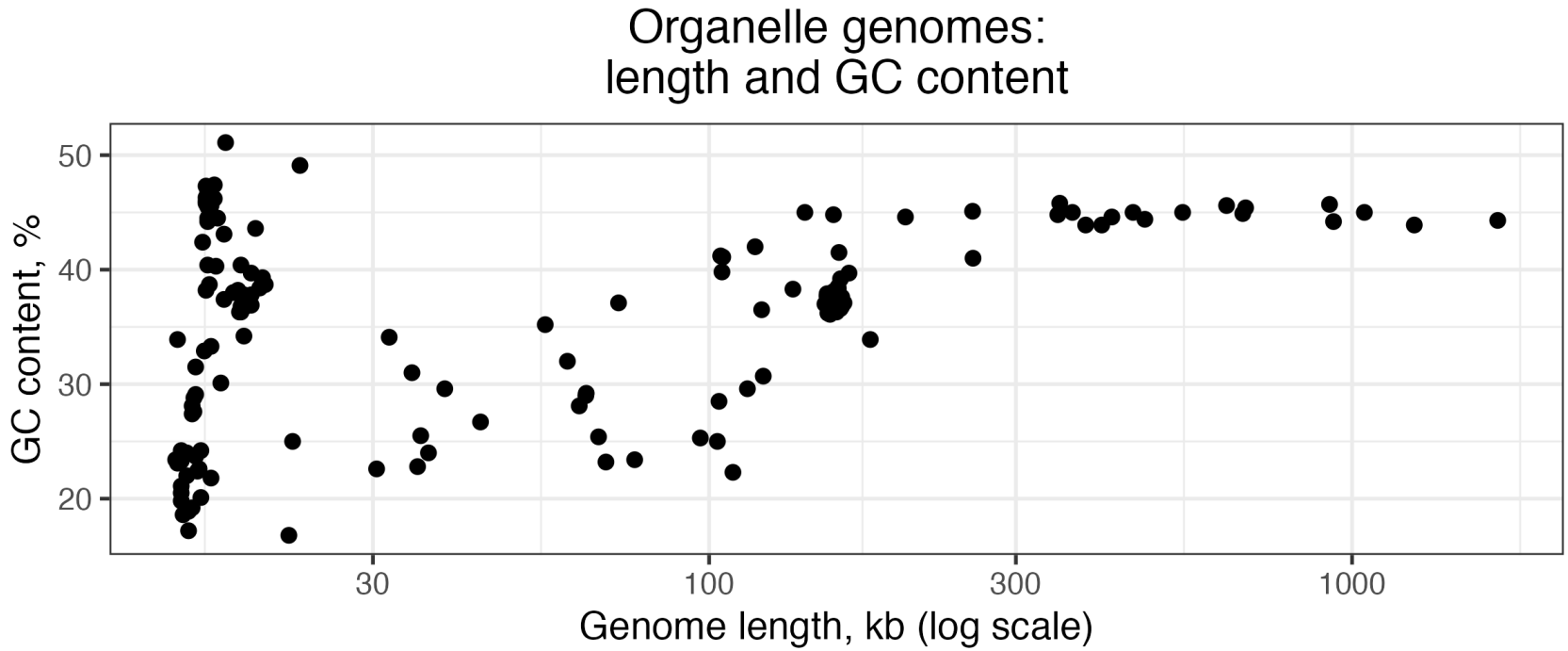


`theme_set()` makes a theme the default for all plots in this R session (`base_size` sets the font size):

```
theme_set(theme_bw(base_size = 14))
```

Axis labels and title

```
ggplot(data = genomes, aes(x = length_kb, y = gc_percent)) +
  geom_point() +
  scale_x_log10() +
  labs(x = "Genome length, kb (log scale)", y = "GC content, %",
       title = "Organelle genomes:\nlength and GC content") +
  theme(plot.title = element_text(hjust = 0.5))  # center the title
```

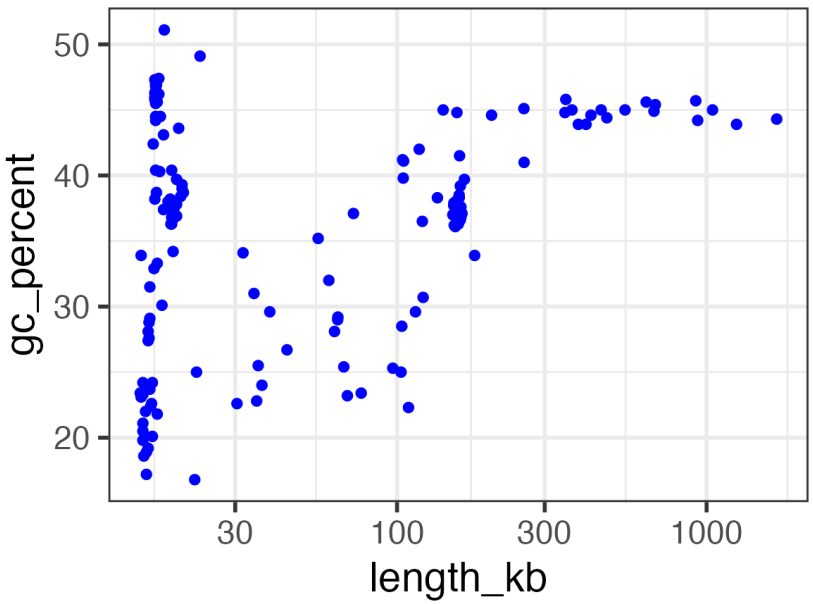


\n starts a new line in a label.

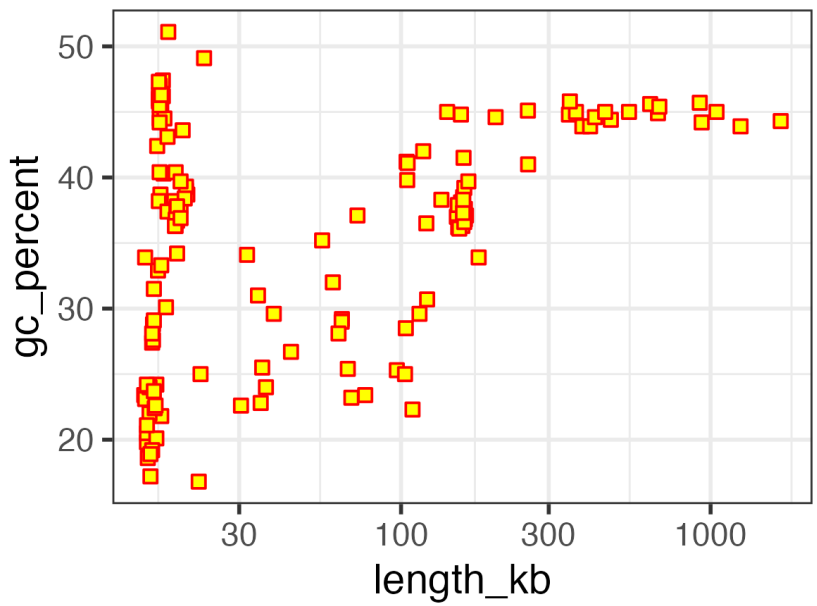
Point size, color and shape

Settings outside aes( ) apply to all points:

```
ggplot(genomes, aes(x = length_kb,
                    y = gc_percent)) +
  geom_point(size = 1,
             color = "blue") +
  scale_x_log10()
```




```
ggplot(genomes, aes(x = length_kb,
                    y = gc_percent)) +
  geom_point(shape = 22, size = 2,
             color = "red",
             fill = "yellow") +
  scale_x_log10()
```



shape is a number from 0 to 25 (`?points`); shapes 21–25 have both an outline (`color`) and a `fill`.

A plot as an object; saving a plot

A plot can be stored in a variable and changed later:

```
gc_plot <- ggplot(genomes, aes(x = length_kb, y = gc_percent)) +
  geom_point() +
  scale_x_log10()
gc_plot <- gc_plot + labs(x = "Genome length, kb (log scale)", y = "GC
↪  content, %")
```

`ggsave()` saves a plot to a file. The file type comes from the extension: `.png` for slides and documents, `.pdf` for print (it stays sharp at any size).

```
ggsave("gc_plot.png", plot = gc_plot, width = 15, height = 10, units =
↪  "cm", dpi = 300)
ggsave("gc_plot.pdf", plot = gc_plot, width = 15, height = 10, units = "cm")
```

The files appear in your project folder and in the `Files` pane.

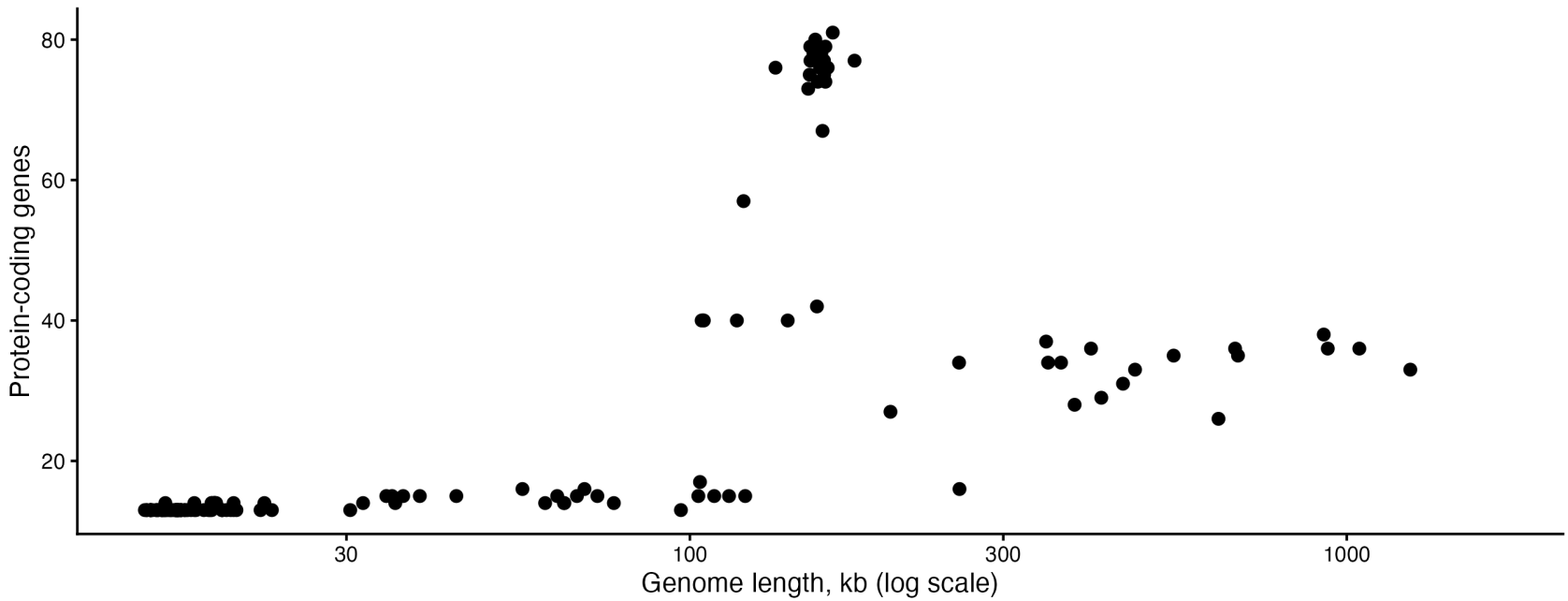
Your turn 2: a scatterplot

1. Draw a scatterplot of the number of protein-coding genes (y) against genome length (x), with genome length on a logarithmic scale.
2. Label the axes "Genome length, kb (log scale)" and "Protein-coding genes".
3. Use `theme_classic()` and points of size 2.
4. Save the plot as `protein_genes.png`, 15 × 10 cm.

R warns that it removed 1 row: the cucumber, with `protein_genes = NA`.

Your turn 2: answers

```
prot_plot <- ggplot(genomes, aes(x = length_kb, y = protein_genes)) +
  geom_point(size = 2) +
  scale_x_log10() +
  labs(x = "Genome length, kb (log scale)", y = "Protein-coding genes") +
  theme_classic()
prot_plot
```



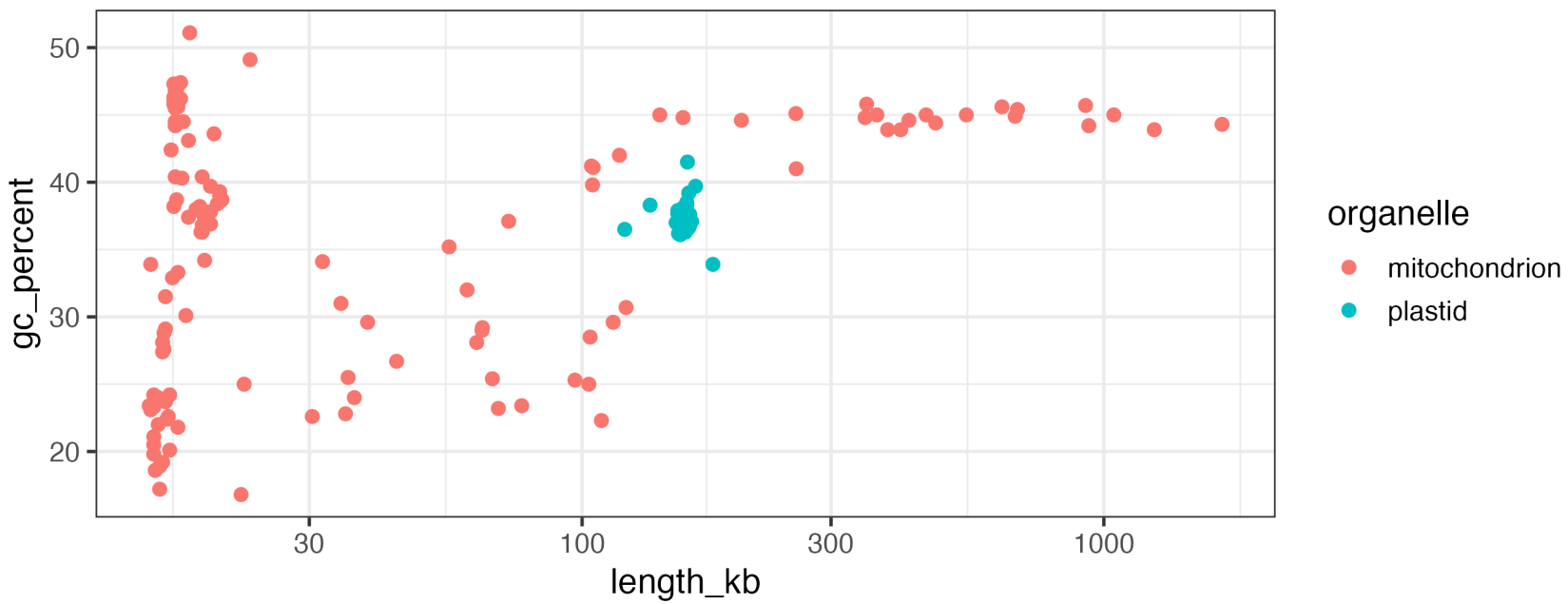

```
ggsave("protein_genes.png", plot = prot_plot, width = 15, height = 10,
↪  units = "cm")
```

Aesthetics

Mapping a variable to color

Inside `aes()`, `color` shows a **variable**: each organelle gets its own color, and a legend appears.

```
ggplot(genomes, aes(x = length_kb, y = gc_percent, color = organelle)) +
  geom_point() +
  scale_x_log10()
```

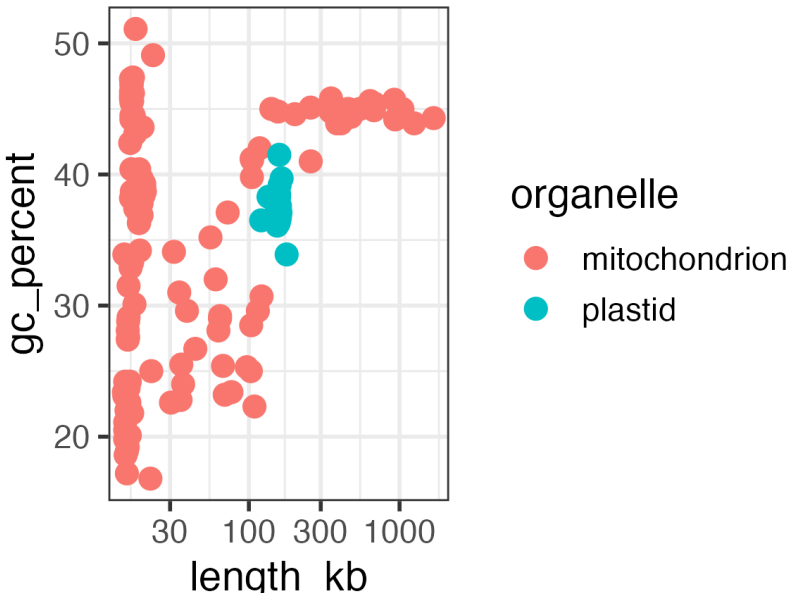


Outside `aes()`, `color = "blue"` is a **setting**: the same color for all points.

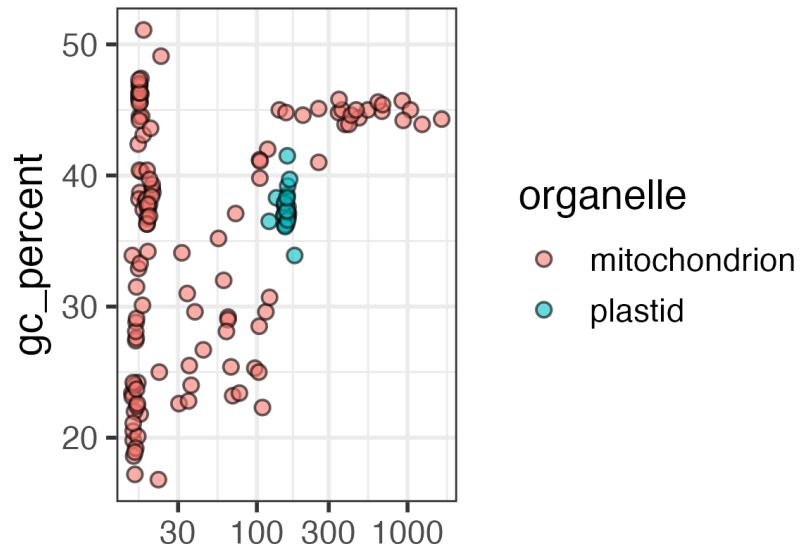
Overlapping points

Many points lie on top of each other (e.g. the plastids), so you cannot see how many there are. Three fixes: smaller points (`size`), see-through points (`alpha`, from 0 to 1), and points with a black outline and a colored `fill` (`shape = 21`):

```
ggplot(genomes, aes(x = length_kb,
                    y = gc_percent,
                    color =
                    ↪  organelle)) +
  geom_point(size = 3) +
  scale_x_log10()
```



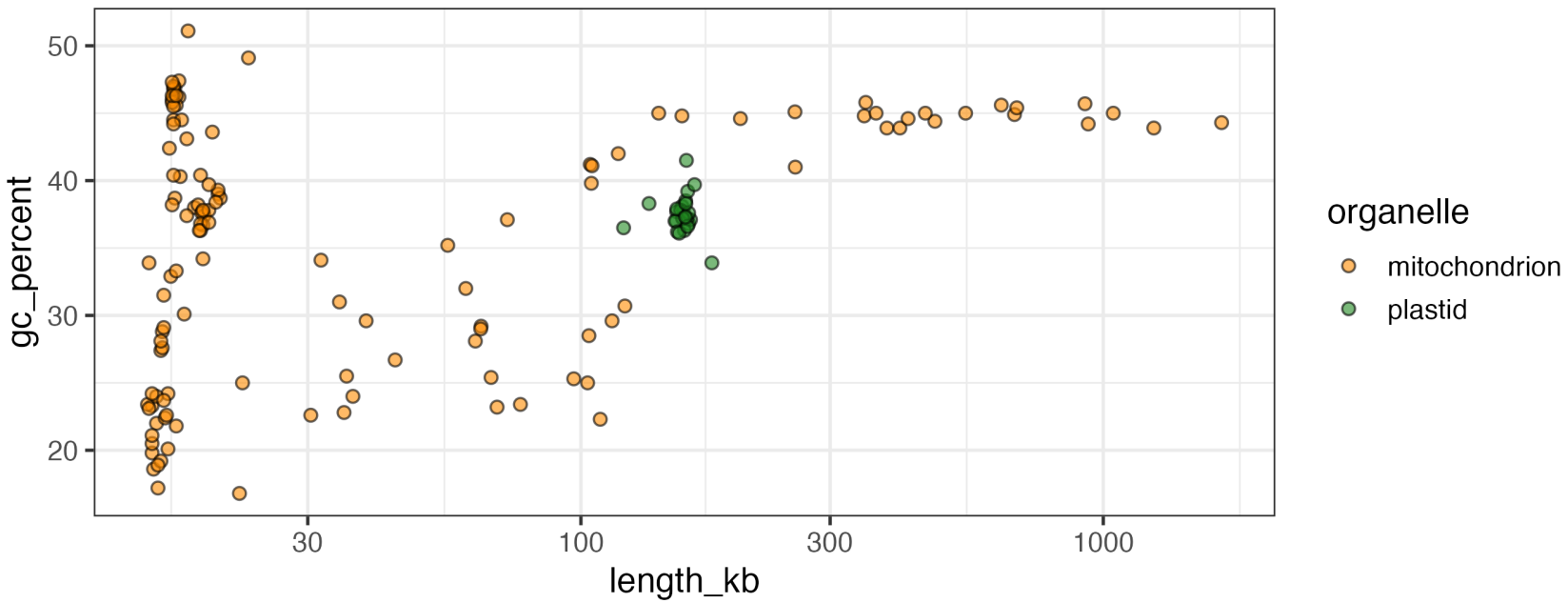

```
ggplot(genomes, aes(x = length_kb,
                    y = gc_percent,
                   fill = organelle)) +
  geom_point(shape = 21,
             color = "black",
             size = 2, alpha = 0.6) +
  scale_x_log10()
```



Choosing your own colors

`scale_fill_manual()` gives one color per group (`scale_color_manual()` does the same for `color`). The colors go to the groups in alphabetical order: `mitochondrion`, then `plastid`.

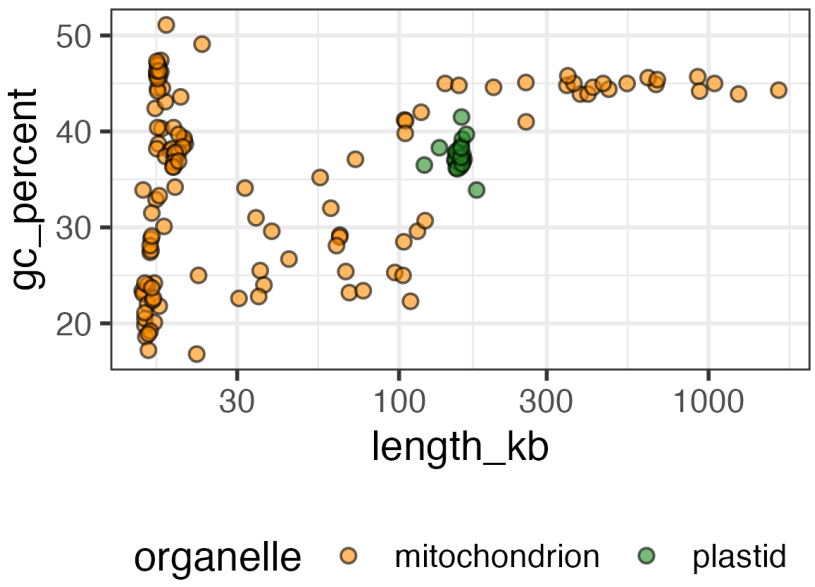
```
org_plot <- ggplot(genomes, aes(x = length_kb, y = gc_percent,
                                fill = organelle)) +
  geom_point(shape = 21, color = "black", size = 2, alpha = 0.6) +
  scale_x_log10() +
  scale_fill_manual(values = c("darkorange", "forestgreen"))
org_plot
```



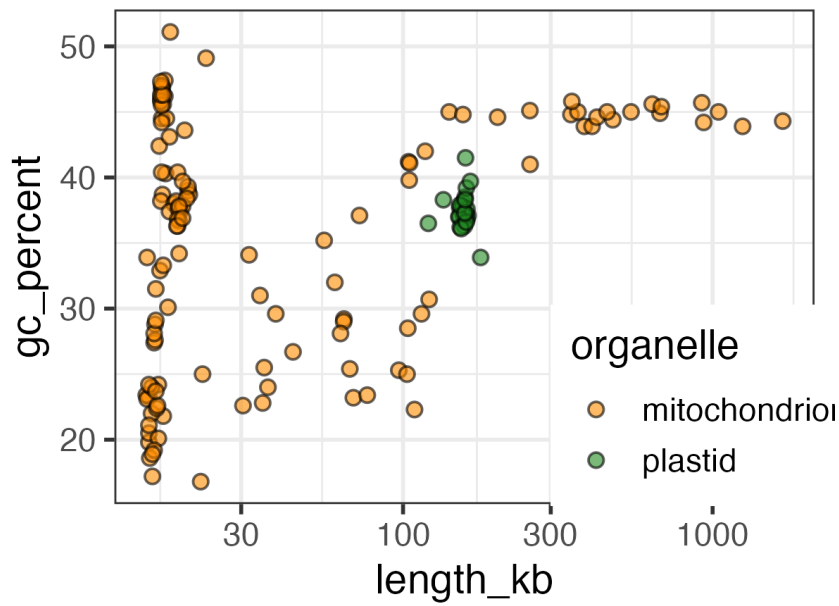
Legend position

`theme(legend.position = ...)`: `"right"` (default), `"bottom"`, `"top"`, `"left"`, `"none"` (no legend), or `"inside"` the plot area:

```
org_plot +
  theme(legend.position = "bottom")
```




```
org_plot +
  theme(legend.position = "inside",
        legend.position.inside =
        ↪  c(0.85, 0.2))
```

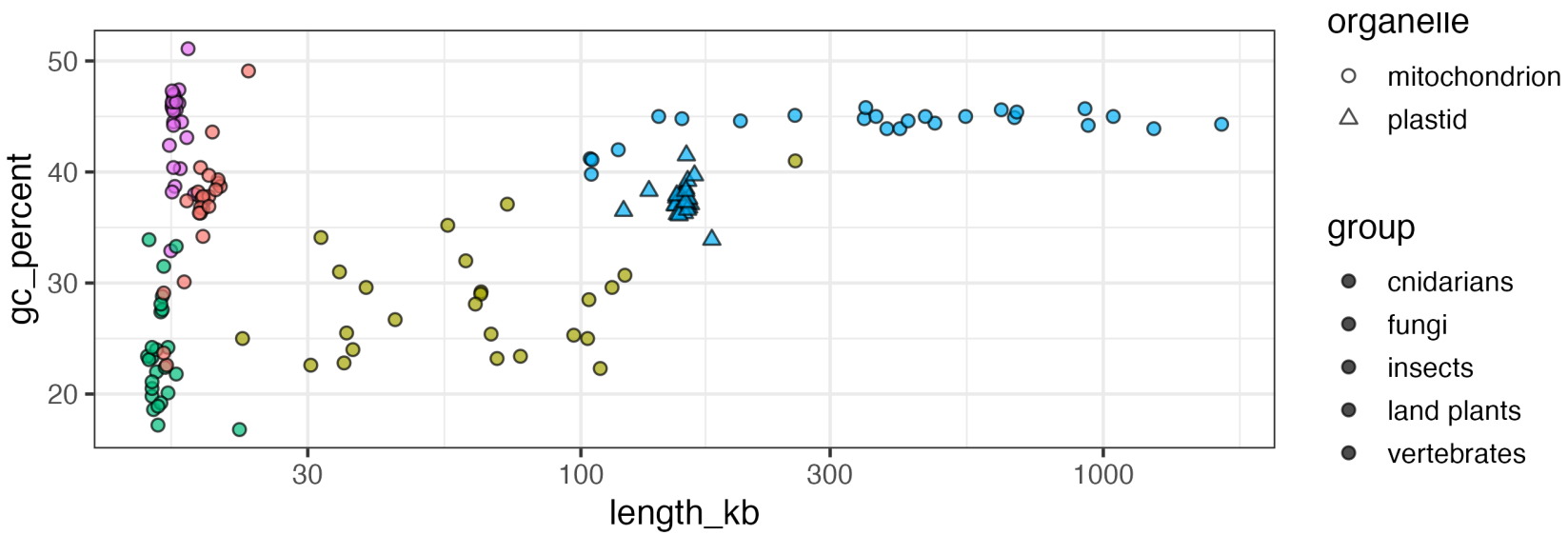


`legend.position.inside = c(0.85, 0.2)`: the position as a fraction of the plot width and height, from the bottom left corner.

Two variables: fill and shape

shape can show a second variable. `scale_shape_manual()` picks shapes that have a fill (21: circle, 24: triangle):

```
ggplot(genomes, aes(x = length_kb, y = gc_percent,
                    fill = group, shape = organelle)) +
  geom_point(color = "black", size = 2, alpha = 0.7) +
  scale_shape_manual(values = c(21, 24)) +
  scale_x_log10()
```



Two legends, 6 combinations: possible, but hard to read. Facets are often better.

Comparing organisms: mitochondria only

To compare organisms, we compare the same kind of genome: mitochondria. Plastids exist only in plants, so a "land plants" group with both organelles would mix two kinds of genomes.

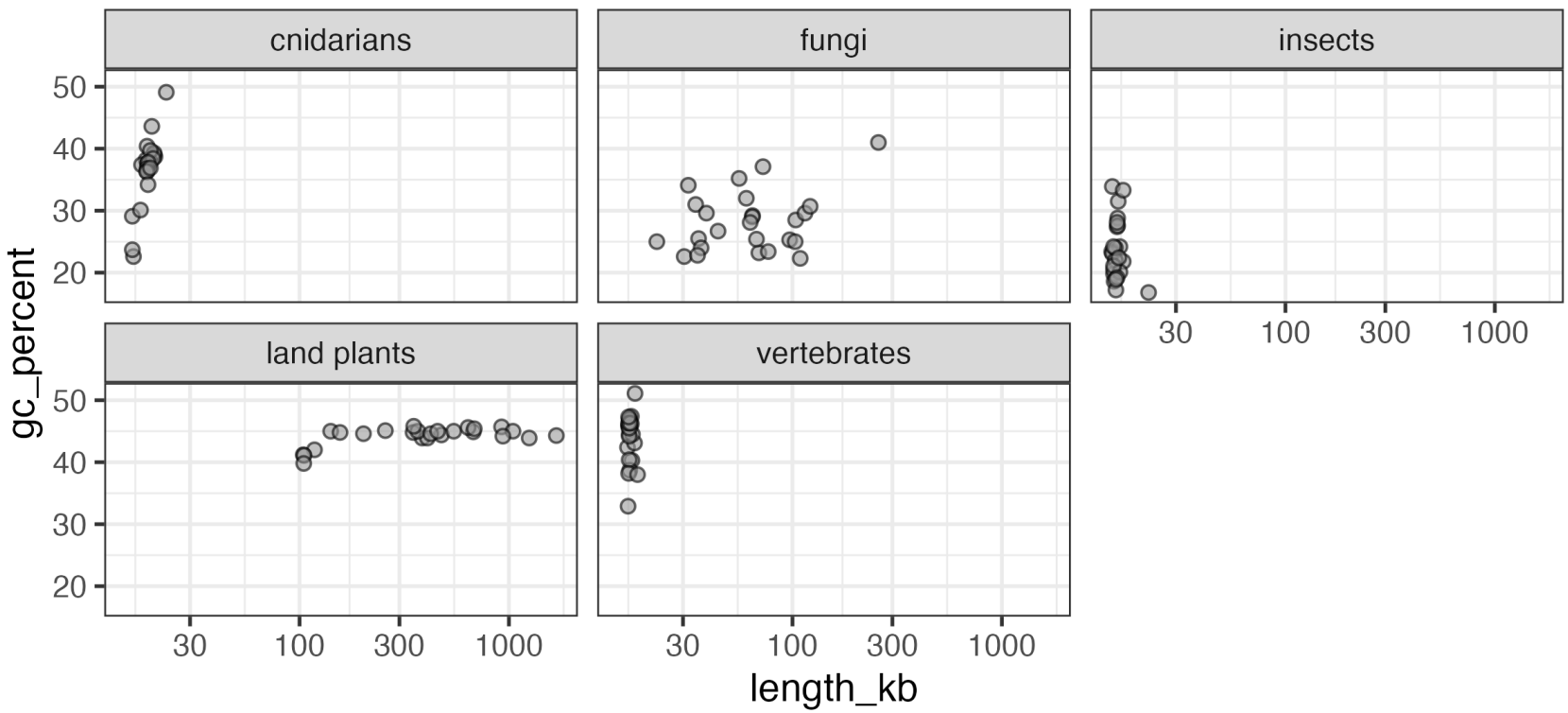
Keep only the rows where `organelle` is `"mitochondrion"`:

```
mito <- genomes[genomes$organelle == "mitochondrion", ]
nrow(mito)
## [1] 125
```

Facets

A **facet** is a small panel with one group of the data. `facet_wrap(~ group)` draws one panel per group, with the same axes:

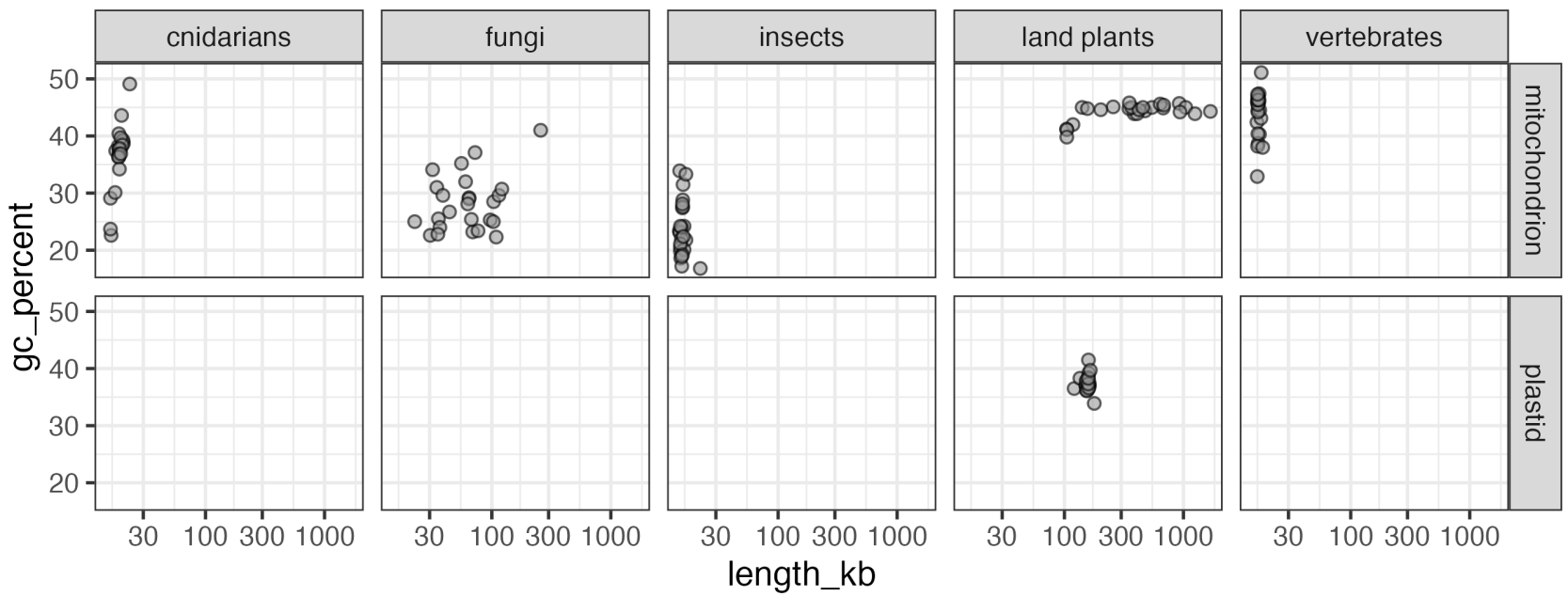
```
ggplot(mito, aes(x = length_kb, y = gc_percent)) +
  geom_point(shape = 21, color = "black", fill = "grey60",
             size = 2, alpha = 0.6) +
  scale_x_log10() +
  facet_wrap(~ group)
```



Facets in a grid

`facet_grid(rows ~ columns)`: one panel for each combination of two variables (here all genomes, both organelles).

```
ggplot(genomes, aes(x = length_kb, y = gc_percent)) +
  geom_point(shape = 21, color = "black", fill = "grey60",
             size = 2, alpha = 0.6) +
  scale_x_log10() +
  facet_grid(organelle ~ group)
```



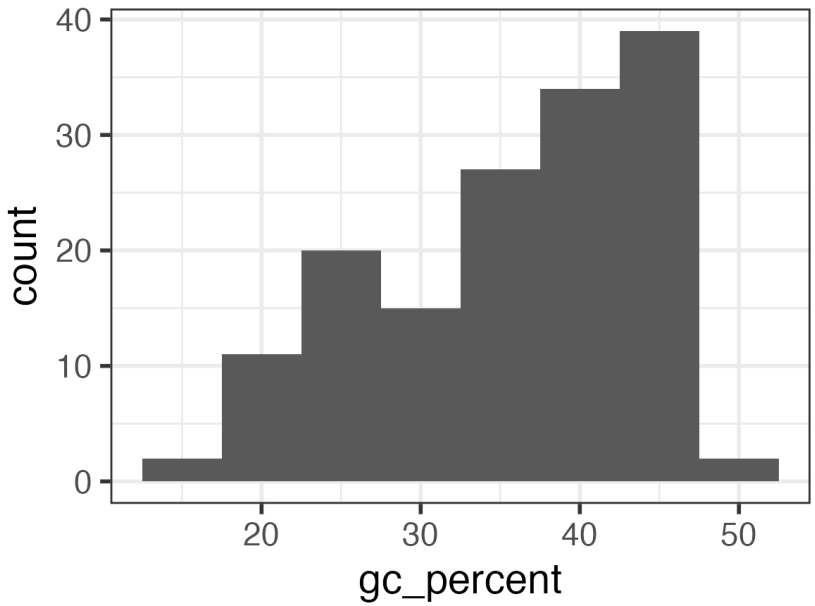
Most panels in the plastid row are empty: only land plants have plastids.

Distributions

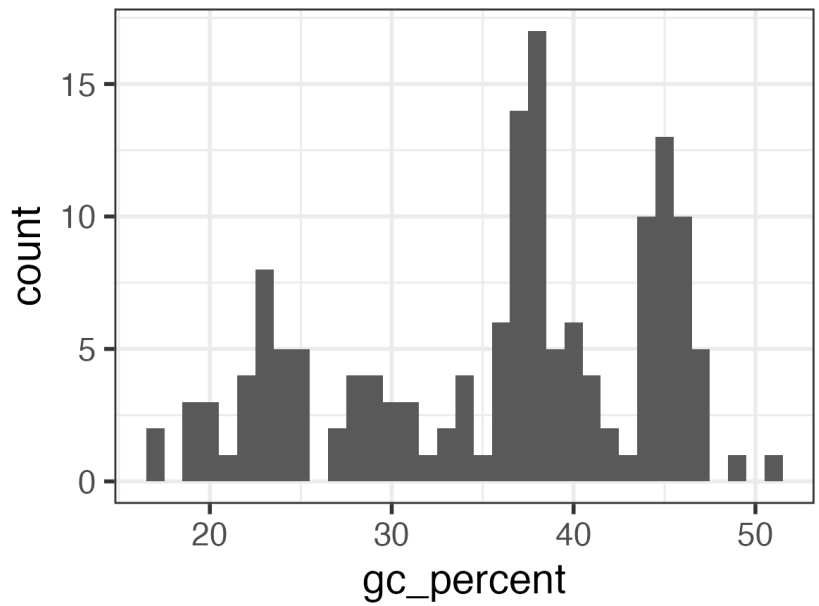
Histogram

A **histogram** shows how often values fall into intervals (bins). `binwidth` sets the width of a bin:

```
ggplot(genomes,
       aes(x = gc_percent)) +
  geom_histogram(binwidth = 5)
```




```
ggplot(genomes,
       aes(x = gc_percent)) +
  geom_histogram(binwidth = 1)
```

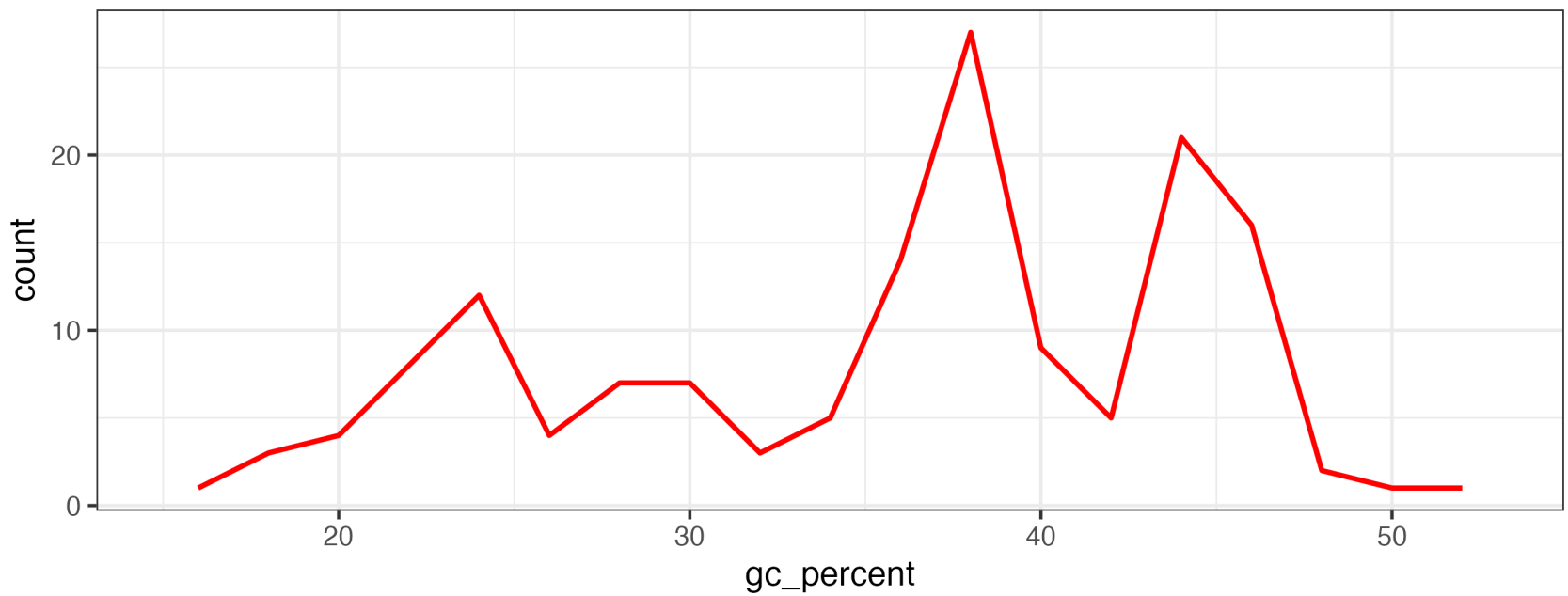


Wide bins hide details; very narrow bins show mostly noise. Try a few widths.

Geoms and stats

A histogram first counts the values in each bin (a statistical step, a "stat") and then draws bars (a geom). The same counts can be drawn by another geom: `stat_bin()` with `geom = "line"`.

```
ggplot(genomes, aes(x = gc_percent)) +
  stat_bin(geom = "line", binwidth = 2, linewidth = 1, color = "red")
```

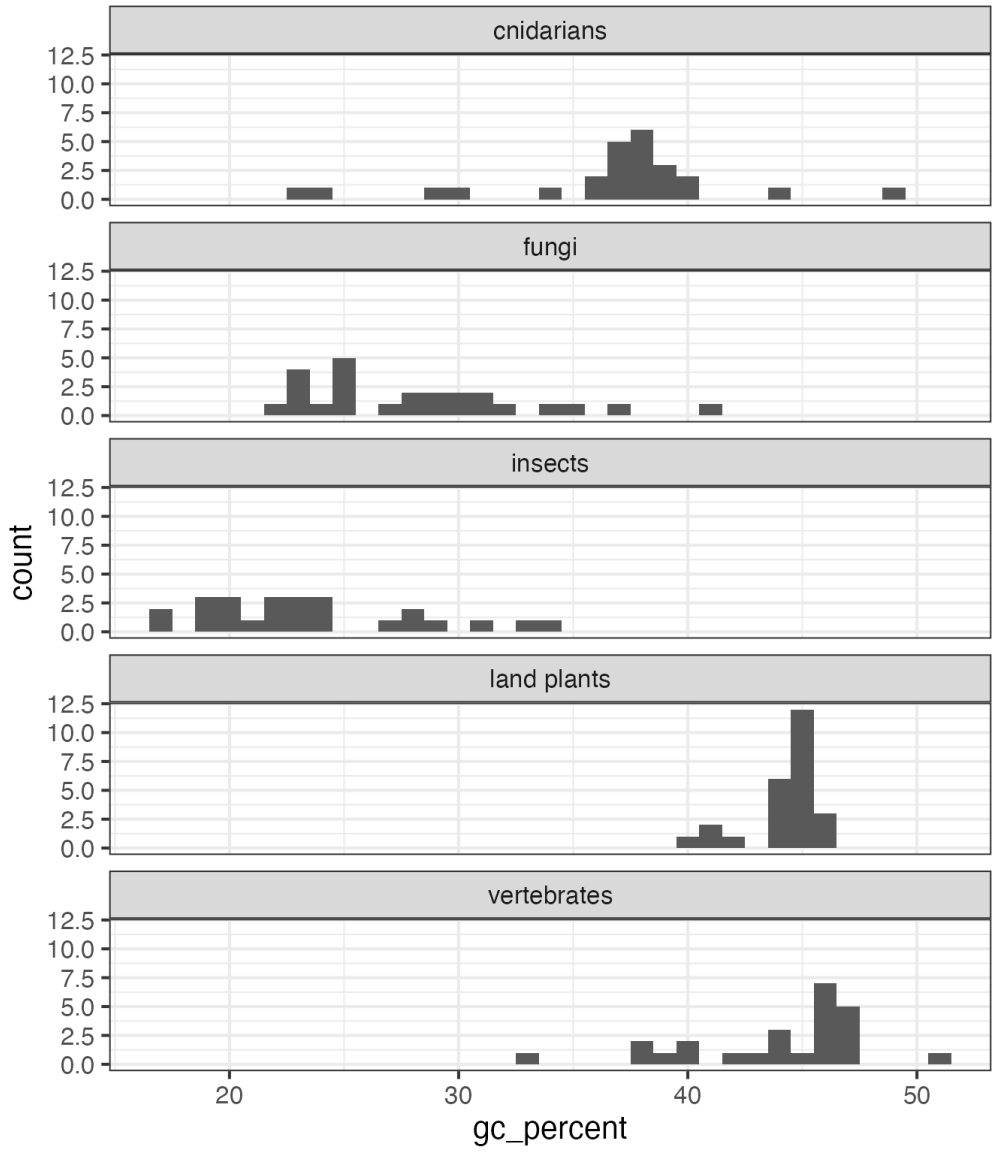


`linewidth` sets the thickness of lines; `size` sets the size of points.

Comparing groups

Facets one above the other (`ncol = 1`) share the x axis, so the groups are easy to compare (mitochondria only):

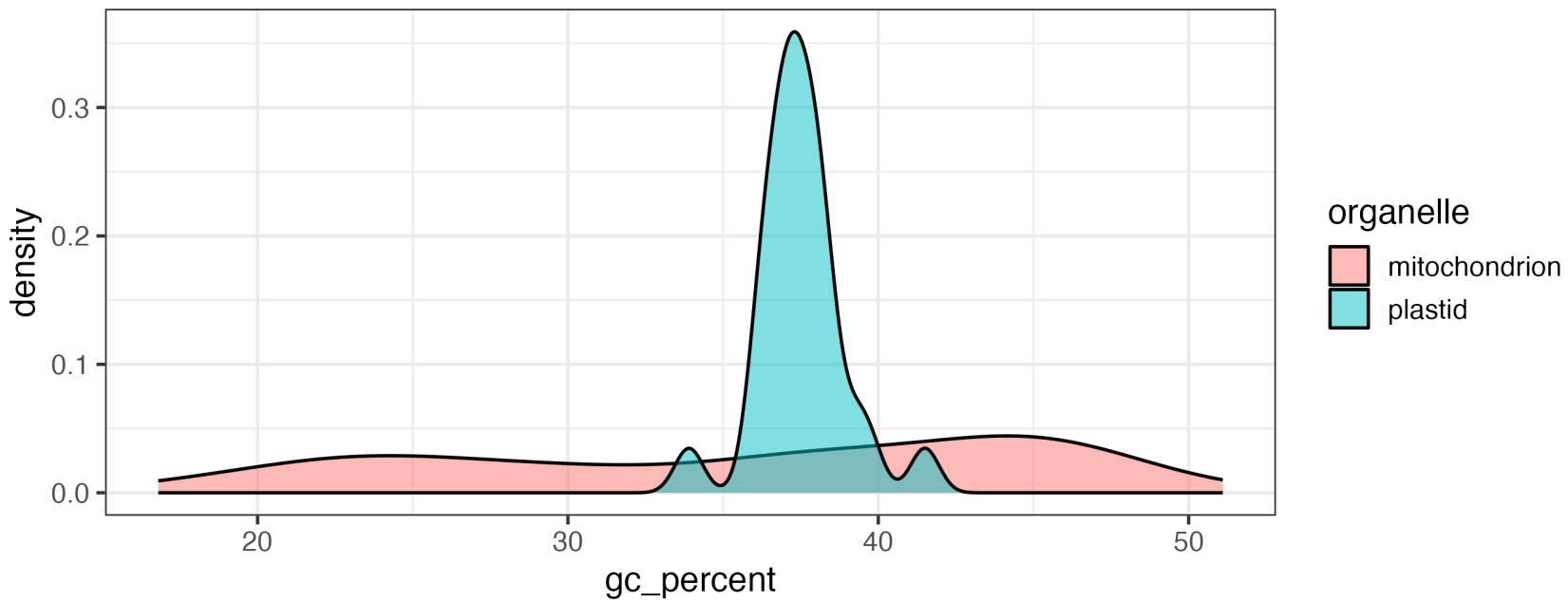
```
ggplot(mito, aes(
  x = gc_percent)) +
  geom_histogram(
    binwidth = 1) +
  facet_wrap(~ group,
             ncol = 1)
```



Density curves

A density curve is a smooth version of a histogram. `fill` colors the area; `alpha` makes it see-through:

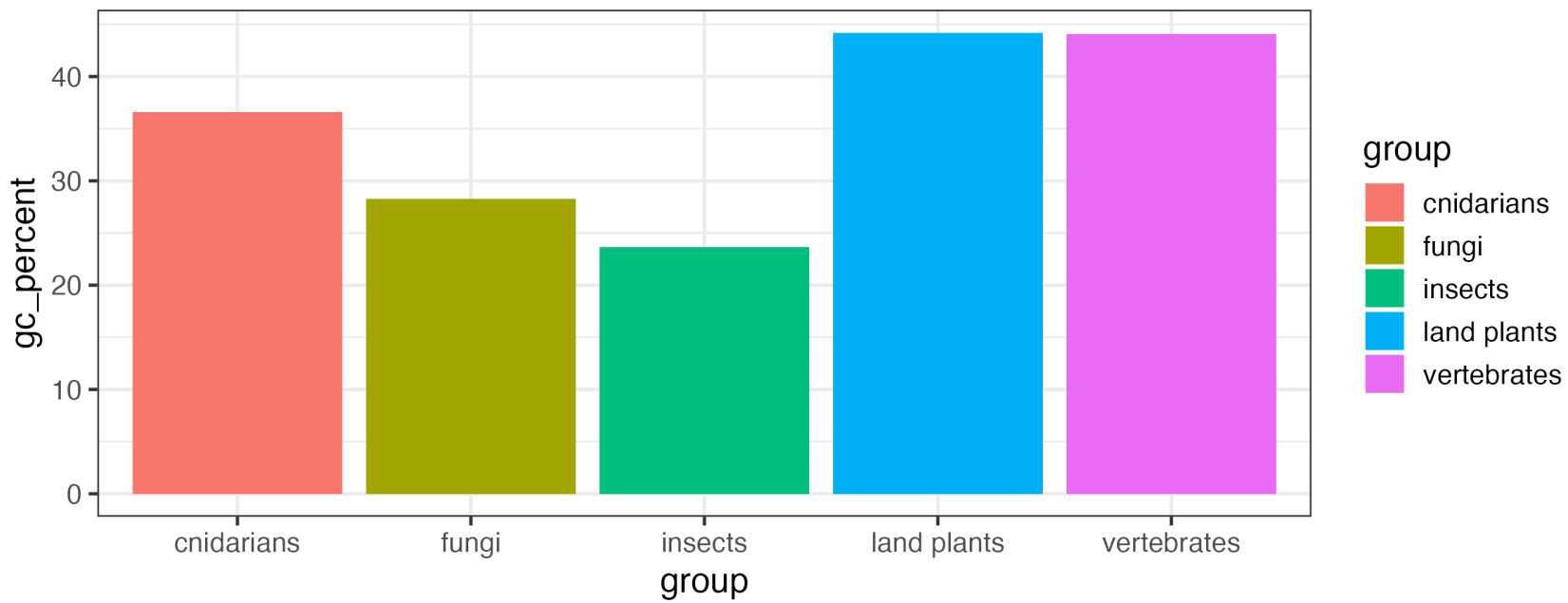
```
ggplot(genomes, aes(x = gc_percent, fill = organelle)) +
  geom_density(alpha = 0.5)
```



Group means

`stat_summary()` calculates a statistic for each group and draws it with a geom; `fun = "mean"` gives the mean:

```
ggplot(mito, aes(x = group, y = gc_percent, fill = group)) +
  stat_summary(fun = "mean", geom = "bar")
```

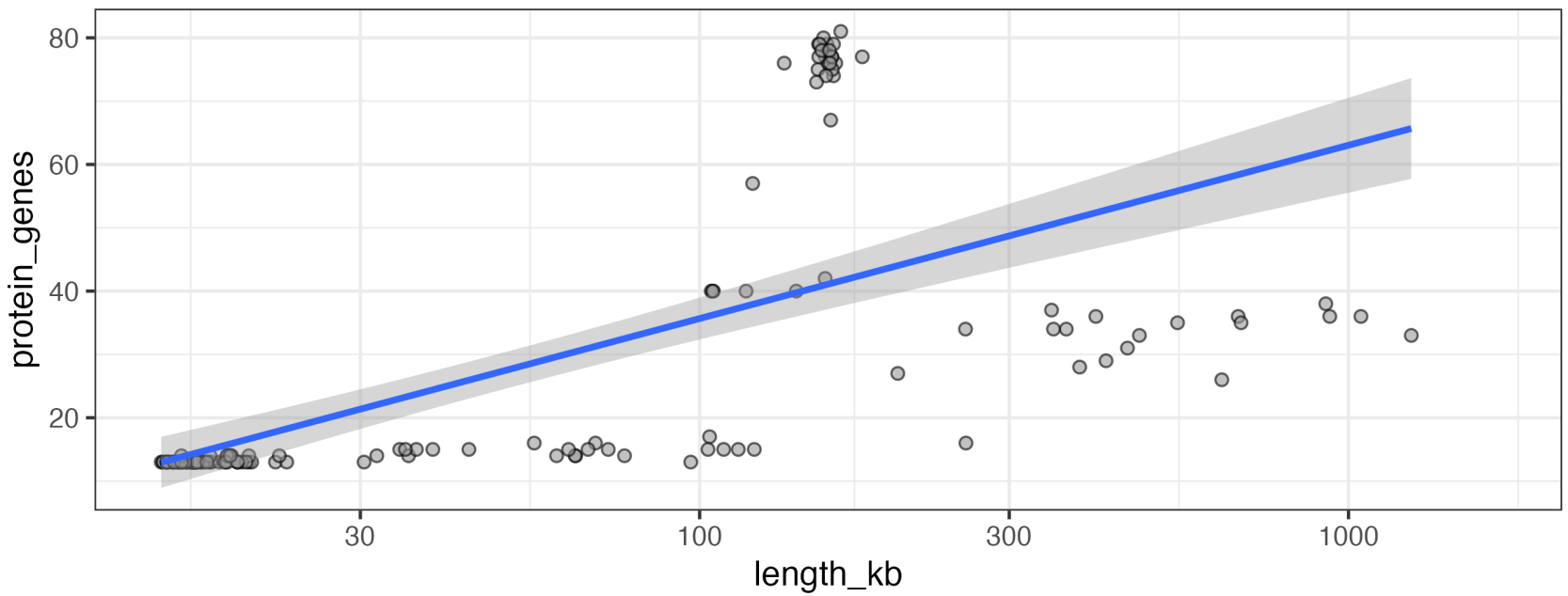


The bars show mean GC%, not mean genome length: lengths are very skewed (3 plant mitogenomes are over 1000 kb), so a mean length would mislead. The median, later today, handles this.

A trend line

`geom_smooth(method = "lm")` adds a straight line fitted to the points: a linear model, the topic of Nov 9.

```
ggplot(genomes, aes(x = length_kb, y = protein_genes)) +
  geom_point(shape = 21, color = "black", fill = "grey60",
             size = 2, alpha = 0.6) +
  scale_x_log10() +
  geom_smooth(method = "lm")
```



Your turn 3: groups and distributions

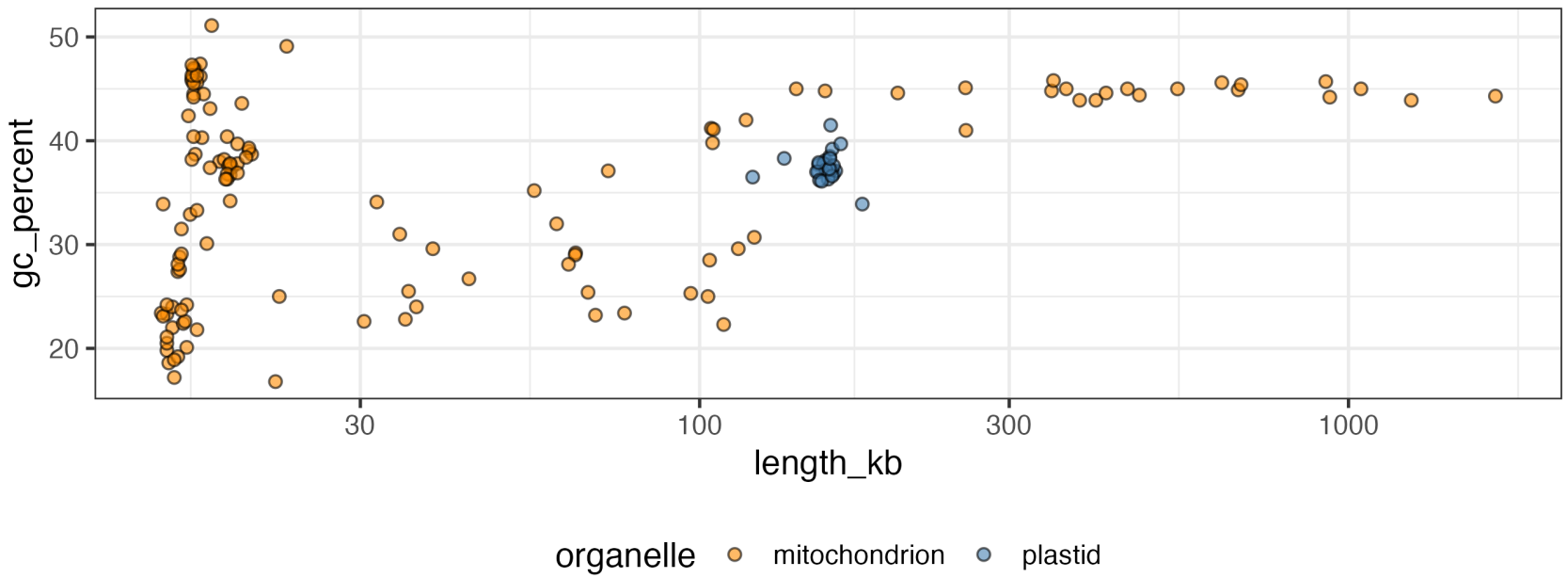
1. Draw GC content (y) against genome length (x, log scale) for all genomes, in the readable style (`shape = 21`, `alpha`), with the fill showing `organelle` and the legend at the bottom. Fill the points with your two favorite colors.

   Hint: `scale_fill_manual()` works like `scale_color_manual()`, but for `fill`. Colors can be names (`"darkorange"`, `"steelblue"`) or hex codes (`"#1B9E77"`). The first color goes to `mitochondrion`, the second to `plastid` (alphabetical order).

2. Draw a histogram of `protein_genes` of the mitochondria (`mito`), with `binwidth = 5` and one facet per `group` in one column.

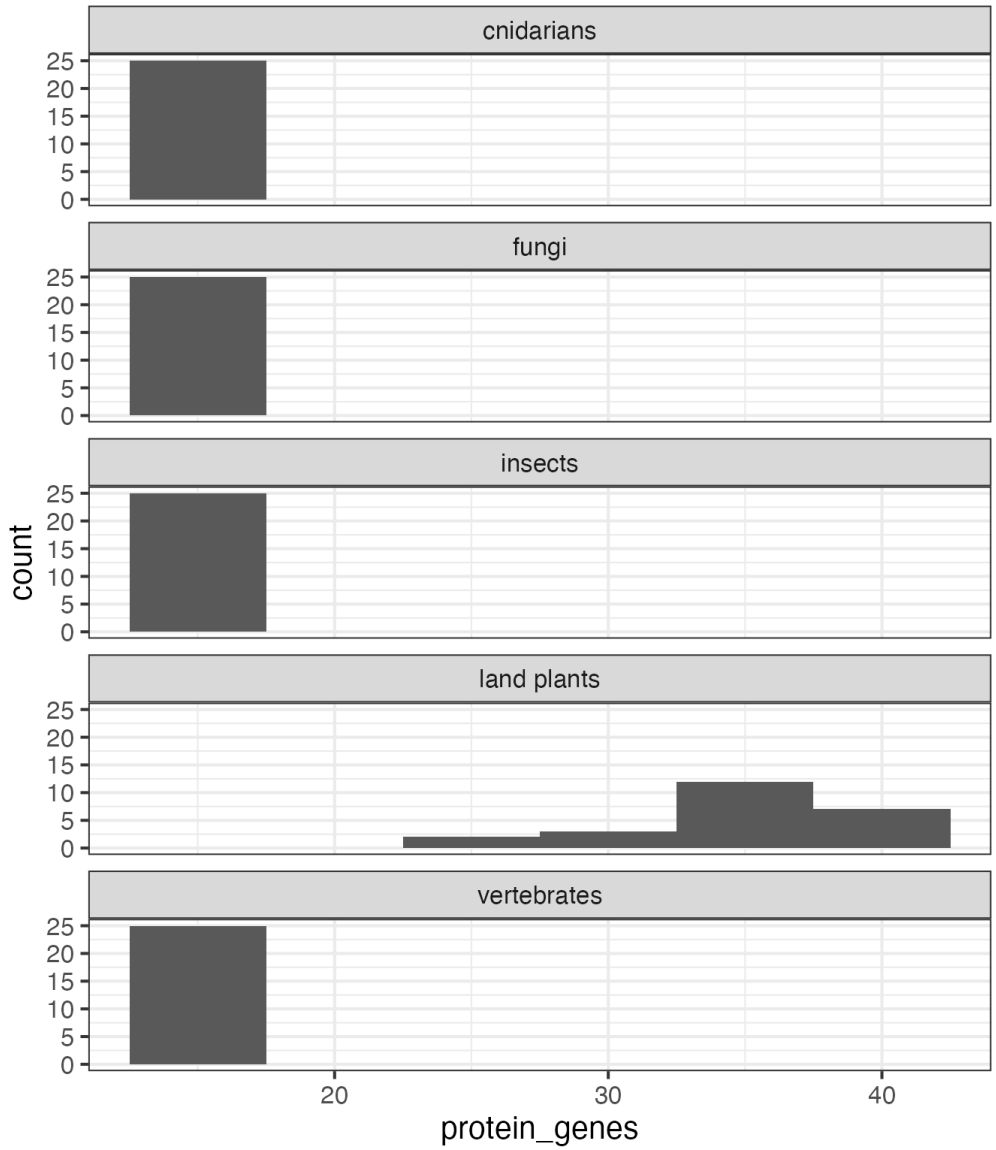
Your turn 3: answers

```
ggplot(genomes, aes(x = length_kb, y = gc_percent, fill = organelle)) +
  geom_point(shape = 21, color = "black", size = 2, alpha = 0.6) +
  # any two colors work:
  scale_fill_manual(values = c("darkorange", "steelblue")) +
  scale_x_log10() +
  theme(legend.position = "bottom")
```



Your turn 3: answers

```
ggplot(mito, aes(
  x = protein_genes)) +
  geom_histogram(
    binwidth = 5) +
  facet_wrap(~ group,
             ncol = 1)
```



Describing a sample

Population and sample

- **Population**: all the objects you want to draw conclusions about, e.g. all insect mitochondrial genomes.
- **Sample**: the objects you actually measured, e.g. our 25 insect mitogenomes (RefSeq has about 5,000).
- **Descriptive statistics** summarise the sample: where its values are (center) and how far they spread.
- Later in the course we will use samples to draw conclusions about populations.

Types of data

Type	Values	In our table
Categorical	categories (groups), no numbers	`group`, `organelle`
Discrete numeric	counts: whole numbers	`protein_genes`, `trna_genes`
Continuous numeric	measurements: any value in a range	`length_kb`, `gc_percent`

Describing numeric data needs two kinds of numbers:

- Center: the **median** (中位数) today, the mean on Nov 6
- Spread: the quartiles today, the standard deviation on Nov 6

Median

The **median** is the middle value of the sorted data. The lengths of the first 8 insect mitogenomes:

```
insects <- mito[mito$group == "insects", ]
sizes <- head(insects$length_kb, 8)
sizes
## [1] 15.9 15.1 15.1 14.8 16.2 14.9 16.8 22.2
sort(sizes)
## [1] 14.8 14.9 15.1 15.1 15.9 16.2 16.8 22.2
```

With an odd number of values, the median is the middle one. With an even number (here 8), it is the mean of the two middle values: the 4th (15.1) and the 5th (15.9).

```
median(sizes)
## [1] 15.5
```

The median is robust to outliers

Suppose the cucumber mitogenome (1684.6 kb) ended up in the insect table by mistake:

```
sizes2 <- c(sizes, 1684.6)
median(sizes)
## [1] 15.5
median(sizes2)
## [1] 15.9
mean(sizes)
## [1] 16.375
mean(sizes2)
## [1] 201.7333
```

One extreme value moves the mean from 16.4 to 201.7 kb, but the median only from 15.5 to 15.9 kb. An extreme value like this is an **outlier** (离群值).

Quantiles

A **quantile** (分位数) splits the sorted data into parts with the same number of values in each:

- the median splits the data into 2 parts;
- the **quartiles** (四分位数) split it into 4 parts: 25% of the values are below Q1, 50% below Q2 (= the median), 75% below Q3;
- the percentiles split it into 100 parts: e.g. 10% of the values are below the 10th percentile.

`quantile()` calculates them; `probs` gives the proportions:

```
quantile(insects$length_kb, probs = c(0.25, 0.5, 0.75))
##  25%  50%  75%
## 15.1 15.5 15.9
quantile(insects$length_kb, probs = c(0.1, 0.9))
##   10%   90%
## 14.98 16.56
```

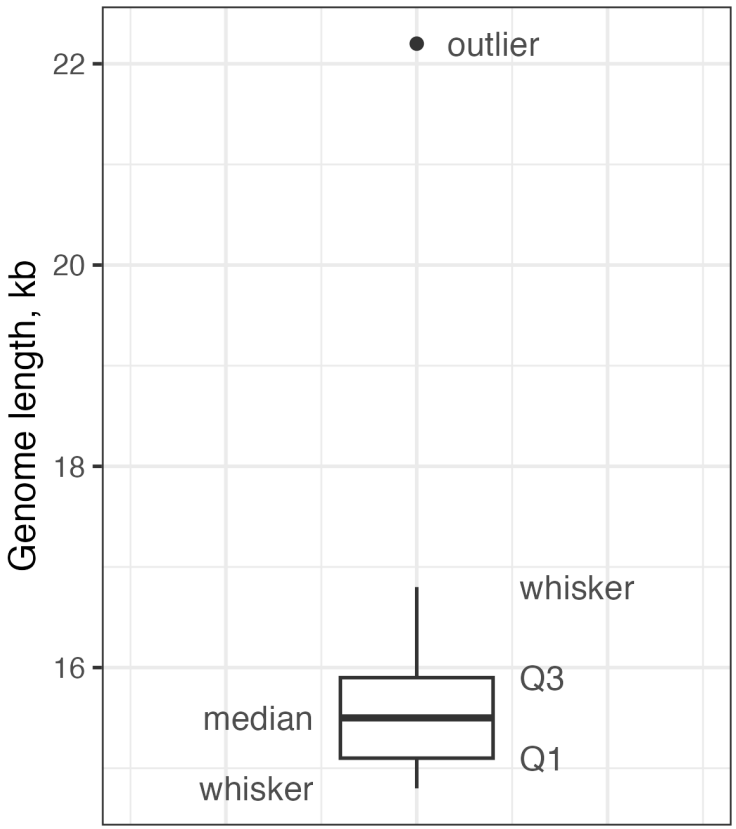
Five-number summary

`quantile()` without `probs` returns the **five-number summary**: minimum, Q1, median, Q3, maximum.

```
quantile(insects$length_kb)
##   0%  25%  50%  75% 100%
## 14.8 15.1 15.5 15.9 22.2
```

Half of the insect mitogenomes are between 15.1 and 15.9 kb long. This range, $Q_3 - Q_1$ = 0.8 kb, is the **interquartile range** (四分位距), IQR for short: the spread of the middle 50% of the values.

Boxplot: the five-number summary as a picture



A **boxplot** (箱形图) of the insect mitogenome lengths:

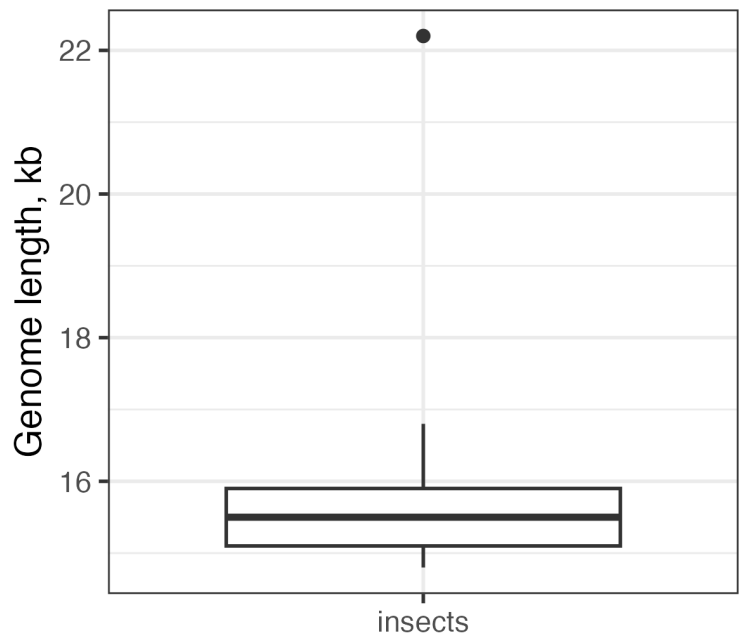
- the line in the box: the median;
- the box: from Q1 to Q3, so its height is the IQR;
- the whiskers: to the most extreme values that are within 1.5 × IQR of the box;
- points beyond the whiskers: possible outliers, drawn one by one (here 1).

The whiskers reach the minimum and the maximum only when there are no outliers.

A boxplot in ggplot2

`geom_boxplot()` needs a numeric variable on y; on x, a group or a text label:

```
ggplot(insects, aes(x = "insects", y = length_kb)) +
  geom_boxplot() +
  labs(x = NULL, y = "Genome length, kb")
```



Summaries by group with the pipe

`dplyr` (part of the tidyverse) calculates summaries for each group:

```
library(dplyr)
```

- The **pipe** `%>%` passes the result on its left into the next function: read it as "and then". `x %>% f()` is the same as `f(x)`.
- `group_by()`: split the table into groups; `summarise()`: one row of summaries per group; `n()`: the number of rows in a group.
- R also has its own pipe, `|>`, which works the same way; in this course we use `%>%`.
- `library(tidyverse)` loads `ggplot2`, `dplyr`, `tidyr` and the other tidyverse packages at once.

Summaries by group: a first example

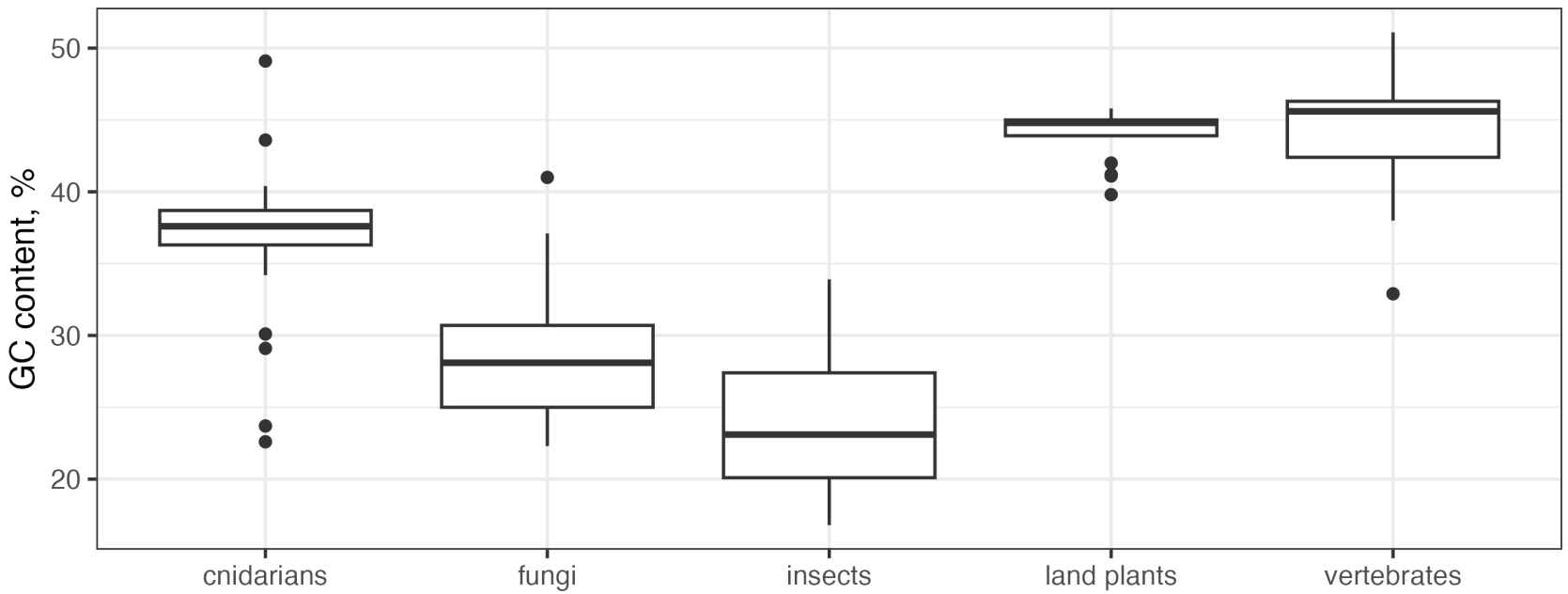
How many genomes of each kind do we have, and how long are they?

```
genomes %>%
  group_by(group, organelle) %>%
  summarise(n = n(), median_length = round(median(length_kb)))
## `summarise()` has regrouped the output.
## i Summaries were computed grouped by group and organelle.
## i Output is grouped by group.
## i Use `summarise(.groups = "drop_last")` to silence this message.
## i Use `summarise(.by = c(group, organelle))` for per-operation
##   grouping (`?dplyr::dplyr_by`) instead.
## # A tibble: 6 x 4
## # Groups:   group [5]
##   group       organelle         n median_length
##   <fct>       <fct>         <int>         <dbl>
## 1 cnidarians  mitochondrion    25            19
## 2 fungi       mitochondrion    25            64
## 3 insects     mitochondrion    25            16
## 4 land plants mitochondrion    25           408
## 5 land plants plastid          25           158
## 6 vertebrates mitochondrion    25            17
```

This is a message, not an error: R tells you the result is still grouped.

Boxplots by group

```
ggplot(mito, aes(x = group, y = gc_percent)) +
  geom_boxplot() +
  labs(x = NULL, y = "GC content, %")
```



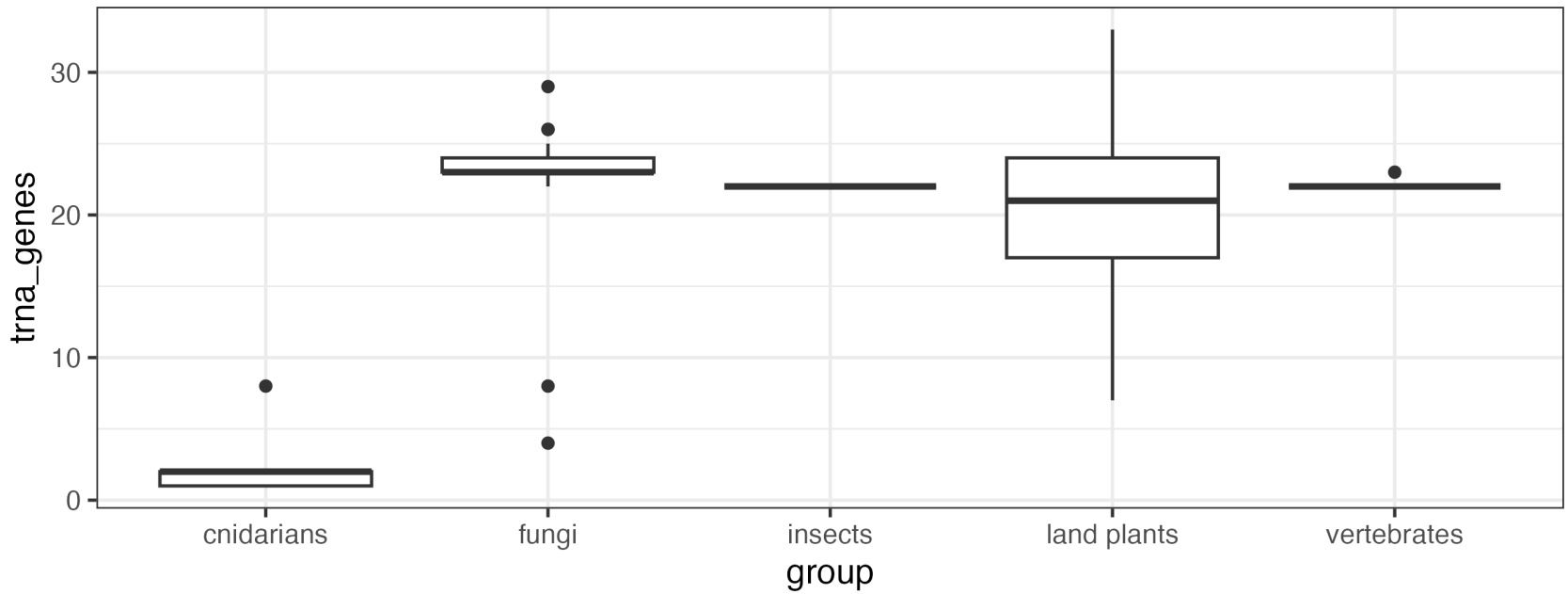
Insect mitogenomes are AT-rich (median GC 23.1%), vertebrate mitogenomes are not (45.6%), although both are about 16 kb long.

Your turn 4: boxplots

1. Draw boxplots of the number of tRNA genes (`trna_genes`) for each `group` of mitochondria (`mito`).
2. Which group has a point far above its box?
3. Calculate the five-number summary of `trna_genes` for each group with `%>%`, `group_by()` and `summarise()`: minimum, Q1, median, Q3, maximum. Use `na.rm = TRUE`.
4. Which group has the largest interquartile range?

Your turn 4: answers

```
ggplot(mito, aes(x = group, y = trna_genes)) +
  geom_boxplot()
```



Cnidarians: one record with 8 tRNA genes (see "An outlier: biology or error?"). The largest IQR: land plants, 7 genes. R warns about the removed NA rows.

Your turn 4: answers

```
mito %>%
  group_by(group) %>%
  summarise(min = min(trna_genes, na.rm = TRUE),
            Q1 = quantile(trna_genes, 0.25, na.rm = TRUE),
            median = median(trna_genes, na.rm = TRUE),
            Q3 = quantile(trna_genes, 0.75, na.rm = TRUE),
            max = max(trna_genes, na.rm = TRUE))
## # A tibble: 5 x 6
##   group         min    Q1 median    Q3   max
##   <fct>       <dbl> <dbl>  <dbl> <dbl> <dbl>
## 1 cnidarians      1     1      2     2     8
## 2 fungi           4    23     23    24    29
## 3 insects        22    22     22    22    22
## 4 land plants     7    17     21    24    33
## 5 vertebrates    22    22     22    22    23
```

With one grouping variable (`group`), `summarise()` gives no message.

An outlier: biology or error?

```
cnid[cnid$trna_genes > 2 & !is.na(cnid$trna_genes), c("organism",
↪  "accession", "trna_genes")]
## # A tibble: 1 x 3
##   organism      accession   trna_genes
##   <chr>         <chr>            <dbl>
## 1 Porites lutea NC_029695.1          8
```

Porites lutea is a stony (scleractinian) coral. The other 6 stony corals in the table have 2 tRNA genes, as is usual for this group. A real gain of tRNA genes, or an error in the annotation?

- Do not delete a point only because it is far from the others: an outlier can be real biology.
- Check the source: open the record (`NC_029695.1` in NCBI Nucleotide) and see how the tRNA genes were found.
- Compare with related species, and keep a note of what you decided: here, `note` says "possible annotation error".

Median and quartiles describe any distribution

- The median and the quartiles do not assume any particular shape of the distribution: they work for symmetric or skewed data, with outliers or without.
- They hardly react to outliers, so they are a safe first summary of any sample.

Summary

Key terms

Term	Meaning
Tidy data	A data table in which each column is one variable, each row is one observation and each cell holds one value. Tidy tables are easy to analyse in R.
Aesthetic	In `ggplot2`, a visual property of the plot that shows a variable from the data: position on the x or y axis, color, shape, size. You set aesthetics inside `aes()`.
Geom	In `ggplot2`, the type of geometric object that draws the data: points (`geom_point()`), bars, lines, boxes.
Facet	In `ggplot2`, one small panel of a plot that shows only one group of the data. Facets put the groups side by side with the same axes.
Logarithmic scale	An axis on which each step multiplies the value by the same number (e.g. 10, 100, 1000) instead of adding to it. It shows small and very large values on one plot.
Median 中位数	The middle value of the sorted data: half of the values are smaller, half are larger. With an even number of values, it is the mean of the two middle values.
Five-number summary	A short description of a sample with five numbers: minimum, first quartile, median, third quartile, maximum.
Boxplot 箱形图	A plot of the five-number summary: the box goes from Q1 to Q3, the line inside is the median, the whiskers show the rest of the data, and outliers are drawn as separate points.

What we have learned

- **Tidy data**: one variable per column, one observation per row, one value per cell. `pivot_longer()` turns a wide table into a long one.
- `read_excel()` reads a sheet; check it with `str()`, `summary()` and `complete.cases()`. A missing value is `NA`, not 0.
- A ggplot is built from layers: data + **aesthetics** (`aes()`) + **geoms**, joined with `+`. Show groups with color, fill, shape or facets; a logarithmic scale shows small and very large values on one axis.
- The **median** and the **quartiles** describe a sample and are robust to **outliers**; the **five-number summary** is shown by a **boxplot**.
- `%>%`, `group_by()` and `summarise()` calculate summaries for each group.

Key R functions: data

Function	What it does	Example
`pivot_longer()`	(`tidyr`) Wide table to long table; `cols`: columns to turn into rows (`-gene`: all except gene); `names_to`, `values_to`: names of the new columns	`pivot_longer(counts_wide, cols = -gene, names_to = "sample", values_to = "count")`
`read_excel()`	(`readxl`) Reads an Excel sheet; `skip`: rows above the header; `na`: code for missing values	`read_excel("data/x.xlsx", skip = 1, na = "NA")`
`is.na()`	TRUE for each missing value	`sum(is.na(genomes$trna_genes))`
`summary()`	Summary of each column, with the number of `NA`'s	`summary(genomes)`
`complete.cases()`	TRUE for each row without `NA`	`genomes[!complete.cases(genomes), ]`
`log10()`	Logarithm to base 10	`log10(100)`

Key R functions: ggplot2

Function	What it does	Example
`ggplot()`, `aes()`	Start a plot: the data and the aesthetics	`ggplot(genomes, aes(x = length_kb, y = gc_percent))`
`geom_point()`	Scatterplot; `shape = 21` + `fill`, `alpha` against overlap	`geom_point(shape = 21, fill = "grey", alpha = 0.6)`
`scale_x_log10()`	Logarithmic x axis	`scale_x_log10()`
`labs()`	Axis labels and title	`labs(x = "Length, kb", title = "...")`
`theme_bw()`, `theme_set()`	A theme; make it the default	`theme_set(theme_bw(base_size = 14))`
`theme()`	Changes parts of the look, e.g. the legend	`theme(legend.position = "bottom")`
`scale_fill_manual()`	Your own colors for groups (`scale_color_manual()` for `color`)	`scale_fill_manual(values = c("red", "blue"))`
`facet_wrap()`, `facet_grid()`	One panel per group	`facet_wrap(~ group, ncol = 1)`
`ggsave()`	Saves a plot to a file	`ggsave("plot.png", plot = p, width = 15, height = 10, units = "cm")`

Key R functions: distributions

Function	What it does	Example
`geom_histogram()`	Histogram; `binwidth` sets the bin width	`geom_histogram(binwidth = 1)`
`stat_bin()`	Counts values in bins, drawn with any geom	`stat_bin(geom = "line", binwidth = 2)`
`geom_density()`	Density curve	`geom_density(alpha = 0.5)`
`stat_summary()`	A statistic for each group, drawn with a geom	`stat_summary(fun = "mean", geom = "bar")`
`geom_smooth()`	A fitted line; `method = "lm"`: a straight line	`geom_smooth(method = "lm")`
`scale_shape_manual()`	Your own shapes for groups	`scale_shape_manual(values = c(21, 24))`

Key R functions: descriptive statistics

Function	What it does	Example
`sort()`	Sorts values from smallest to largest	`sort(sizes)`
`median()`	Median; `na.rm = TRUE` ignores NA	`median(x, na.rm = TRUE)`
`quantile()`	Quantiles; without `probs`: the five-number summary	`quantile(x, probs = c(0.25, 0.75))`
`geom_boxplot()`	Boxplot	`ggplot(mito, aes(x = group, y = gc_percent)) + geom_boxplot()`
`%>%`	Pipe: passes the result on to the next function	`genomes %>% group_by(group)`
`group_by()`	(`dplyr`) Splits a table into groups	`group_by(group, organelle)`
`summarise()`	(`dplyr`) One row of summaries per group	`summarise(median = median(gc_percent))`
`n()`	(`dplyr`) Number of rows in a group, inside `summarise()`	`summarise(n = n())`

Homework and next session

- Moodle quiz on today's session: complete it before Nov 6.
- Optional tasks for bonus points: submit as an `.R` file.
- Nov 6: descriptive statistics II (normal distribution, mean and standard deviation, z-scores, Q-Q plots) and hypothesis testing.